

The RSA Cryptosystem: Foundations, Attacks, and Implementation

Dr. Faisal Aslam

Contents

1	Lesson 1: The RSA Cryptosystem I — Foundations, Correctness, and Factoring	3
1.1	Refresher: The Two Theorems RSA Is Built On	4
1.1.1	The Actual Problem, Before Any Formulas	4
1.1.2	Euler’s Totient Function: Counting the “Well-Behaved” Numbers	5
1.1.3	The Chinese Remainder Theorem	8
1.2	RSA Construction	11
1.2.1	Key Generation	12
1.2.2	Fast Modular Exponentiation	13
1.2.3	CRT-Based Decryption Speedup	14
1.3	Correctness of RSA	15
1.4	Security Foundations	16
1.4.1	The RSA Problem and the RSA Assumption	16
1.4.2	Why Textbook RSA Is Not Secure As-Is	16
1.5	Practical Key Generation	18
1.5.1	Primality Testing	18
1.5.2	Choosing e and d	18
1.5.3	Prime Generation Pitfalls	18
1.6	Factoring Algorithms	20
1.6.1	Trial Division and Fermat’s Method	20
1.6.2	Pollard’s ρ Algorithm	20
1.6.3	Pollard’s $p - 1$ Algorithm	21
1.6.4	Sieve Methods: Quadratic Sieve and GNFS (Overview)	22
1.6.5	Factoring Records and Key-Size Recommendations	22
1.7	Summary and Bridge to Lesson 2	24
2	Lesson 2: The RSA Cryptosystem II — Mathematical Attacks	25
2.1	Wiener’s Attack: Small Private Exponent	26
2.1.1	Motivation	26
2.1.2	Continued Fractions Refresher	26
2.1.3	Wiener’s Theorem	27
2.2	Håstad’s Broadcast Attack	30
2.2.1	Setup	30
2.3	Common Modulus Attack	32
2.3.1	Setup	32
2.4	Cycling Attack (Conceptual)	34
2.4.1	Setup	34
2.5	The Franklin–Reiter Related-Message Attack	36
2.5.1	Setup	36
2.6	Coppersmith’s Method: An Overview	38
2.6.1	The General Problem	38
2.6.2	The Reduction to Lattices	38
2.7	Summary and Bridge to Lesson 3	41

3 Lesson 3: The RSA Cryptosystem III — Implementation and Protocol Attacks	43
3.1 Padding Oracles: Bleichenbacher’s Attack on PKCS#1 v1.5	44
3.1.1 PKCS#1 v1.5 Encryption Padding	44
3.1.2 The Attack, at a High Level	45
3.2 Timing Attacks	46
3.2.1 The Leakage Source	46
3.2.2 Defense: Blinding	47
3.3 Fault Attacks: The Bellcore Attack on CRT-RSA	48
3.3.1 Setup	48
3.4 ROCA: When Key Generation Itself Is the Vulnerability	50
3.4.1 Background	50
3.5 Side-Channel Countermeasures: A Consolidated Checklist	52
3.6 OAEP: A Security Sketch	53
3.6.1 Construction	53
3.7 Guillou–Quisquater: An RSA-Based Zero-Knowledge Proof	55
3.7.1 Motivation: Proving Identity Without Revealing the Secret	55
3.8 Summary	58

Lesson 1: The RSA Cryptosystem I — Foundations, Correctness, and Factoring

Euler's Theorem, the Chinese Remainder Theorem, Key Generation, and Classical Factoring Algorithms

Roadmap

- (1) Refresher: Euler's theorem and the Chinese Remainder Theorem (the two facts RSA is built on)
- (2) RSA construction: key generation, encryption, decryption
- (3) Correctness proof (the full argument, including the edge case textbooks skip)
- (4) Security foundations: the RSA problem, the factoring problem, and how they relate
- (5) Practical key generation: primality testing, choosing e , avoiding weak primes
- (6) Factoring algorithms: trial division, Fermat, Pollard's ρ , Pollard's $p - 1$, sieve methods
- (7) Key-size recommendations and factoring records

What this lecture deliberately excludes: padding schemes, Bleichenbacher's attack, Wiener's attack, Coppersmith's method, fault attacks. These require the machinery built here and are covered in Lessons 2–3.

1.1 Refresher: The Two Theorems RSA Is Built On

Before we define RSA, let's get one thing completely straight: *what problem is RSA actually trying to solve?* Everything in this section is in service of answering that one question, so we start there instead of jumping straight into definitions.

1.1.1 The Actual Problem, Before Any Formulas

Ordinary encryption is easy to design if you only need to scramble a message once: shift every letter by 3, say. The hard part is **decryption** — undoing the scramble — and specifically, doing so *without* the two parties ever having met to agree on a shared secret in advance. RSA's answer is beautifully simple to state and much less simple to justify:

- To **encrypt** a message m (thought of as a number), raise it to some public power: $c = m^e$.
- To **decrypt**, raise the ciphertext to a different, secret power: c^d .

For this to work as a cryptosystem, we need one very specific mathematical guarantee:

$$(m^e)^d = m^{ed} \quad \text{must always equal } m.$$

That's it. That's the entire mathematical content RSA needs to be correct. Everything in this refresher section exists to answer one question: *how do we choose e and d so that m^{ed} always comes back around to m ?* Ordinary arithmetic gives no such guarantee — m^{ed} just grows, it doesn't loop back to m . The trick is to do all of this arithmetic **modulo** some number n instead of over the ordinary integers, because modular arithmetic has a property ordinary arithmetic lacks: it *wraps around*, like a clock. Euler's theorem is the precise statement of *how far you have to raise a number's power before it wraps all the way back to where it started*. That is the entire punchline we are building toward in this section — keep it in mind, because it is easy to get lost in definitions and forget what they are for.

A 60-second reminder of modular arithmetic

If you already know what " $a \equiv b \pmod{n}$ " means, skip ahead. If not: think of a clock face with n numbers on it, $0, 1, \dots, n-1$. Counting forward past $n-1$ wraps back around to 0. " $a \bmod n$ " just means: the position on this n -hour clock that you land on after counting a steps forward from 0. Two numbers a, b are **congruent mod n** , written $a \equiv b \pmod{n}$, if they land on the *same* clock position — equivalently, if n divides $a - b$ exactly. All of ordinary addition, subtraction, and multiplication still work perfectly well on the clock: you can add, subtract, or multiply two clock positions and it doesn't matter whether you reduce mod n before or after doing the arithmetic, you land in the same place either way. This "wraps around and still behaves like normal arithmetic" property is the single fact that makes everything below possible.

Why a clock, and not just "big numbers"? Because a clock is exactly the kind of structure where raising something to higher and higher powers is *guaranteed* to eventually come back to where it started — there are only finitely many positions on the clock, so the sequence $a, a^2, a^3, \dots \pmod{n}$ must eventually repeat a value, and once modular exponentiation repeats a value it cycles forever after. Euler's theorem tells us *exactly which exponent* is guaranteed to land back on 1 — not just eventually, but at a specific, computable, and controllable point. That specific point is what RSA is built around.

1.1.2 Euler's Totient Function: Counting the “Well-Behaved” Numbers

Here's the question that gets us there. Fix a modulus n . Some numbers, when you keep multiplying them together mod n , behave completely predictably and reversibly — you can always undo a multiplication by them. Other numbers behave badly and lose information once you multiply by them mod n (think of multiplying by an even number mod an even n : you can never fully recover what you multiplied). **Euler's totient function**, written $\varphi(n)$, simply counts how many numbers in $\{1, \dots, n\}$ are the “well-behaved,” reversible kind. The formal condition for being “well-behaved” turns out to be exactly “sharing no common factor with n ” — we'll see why in a moment — so that becomes the definition:

Definition 1.1.1 (Euler's totient function). For $n \in \mathbb{Z}^+$, define

$$\varphi(n) = |\{a \in \{1, \dots, n\} : \gcd(a, n) = 1\}|,$$

i.e. $\varphi(n)$ counts how many integers in $\{1, \dots, n\}$ share no common factor with n other than 1 (such integers are called **coprime to n** , or **relatively prime to n**).

Example 1.1.2 (Computing φ by brute force, to build intuition). Take $n = 10$. Check every integer from 1 to 10 against $\gcd(\cdot, 10)$:

a	1	2	3	4	5	6	7	8	9	10
$\gcd(a, 10)$	1	2	1	2	5	2	1	2	1	10

The values with $\gcd(a, 10) = 1$ are $\{1, 3, 7, 9\}$ — exactly 4 of them, so $\varphi(10) = 4$. Notice $\varphi(10)$ is not simply “10 minus something obvious”; it depends on which primes divide 10 (here 2 and 5), not just how many divisors 10 has.

Proposition 1.1.3 (Multiplicativity and formula). If $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ is the prime factorization of n , then

$$\varphi(n) = n \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right).$$

In particular, for two distinct primes p, q :

$$\varphi(pq) = (p-1)(q-1).$$

This special case is the one RSA uses directly.

Remark 1.1.4 (Where this formula comes from, intuitively). Think inclusion–exclusion. Among any p_1 consecutive integers, exactly one is divisible by p_1 , so a $\left(1 - \frac{1}{p_1}\right)$ fraction survives once multiples of p_1 are discarded. The same holds independently for $p_2, \dots, p_k$, and because the p_i are distinct primes, being “divisible by p_i ” and being “divisible by p_j ” turn out not to interfere with each other across $\{1, \dots, n\}$, so the surviving fraction after excluding every prime factor is the product of the individual surviving fractions. Multiplying by n gives the formula. The RSA-relevant case $\varphi(pq) = (p-1)(q-1)$ says the same thing very concretely: from $\{1, \dots, pq\}$, throw away the q multiples of p and the p multiples of q (these two lists overlap only at pq itself, discarded either way), leaving $pq - p - q + 1 = (p-1)(q-1)$ survivors.

Example 1.1.5 (Computing φ with the formula). $36 = 2^2 \cdot 3^2$, so $\varphi(36) = 36 \left(1 - \frac{1}{2}\right) \left(1 - \frac{1}{3}\right) = 36 \cdot \frac{1}{2} \cdot \frac{2}{3} = 12$. Direct check: $\{1, 5, 7, 11, 13, 17, 19, 23, 25, 29, 31, 35\}$ are exactly the 12 residues coprime to 36 — none of them divisible by 2 or by 3.

For the RSA-relevant case: $p = 61, q = 53$ gives $\varphi(61 \cdot 53) = 60 \cdot 52 = 3120$ — this is the pair we will reuse throughout this lecture.

Why we care: the coprime numbers form a self-contained “club”

Here is the fact that makes $\varphi(n)$ worth naming at all, explained slowly, with no jargon first, and then with the standard vocabulary attached afterward.

Claim: if you take two numbers that are each coprime to n , and multiply them mod n , the result is *again* coprime to n . In words: multiplying two “well-behaved” numbers mod n never produces a “badly-behaved” one. This is not obvious from nothing, but it is easy to see why it’s true: if a and n share no prime factor, and b and n share no prime factor, then ab cannot suddenly acquire a prime factor of n out of nowhere either — multiplication doesn’t create new prime factors that weren’t already in a or b . So the set of numbers coprime to n is **closed** under multiplication mod n : start inside the set, multiply, you stay inside the set.

That alone would already be interesting, but there is more: every number coprime to n also has a genuine *multiplicative inverse* mod n — another number in the same set that, when multiplied by it, gives 1. (This is exactly what the Extended Euclidean Algorithm computes, and it is guaranteed to exist precisely because $\gcd(a, n) = 1$.) So this set of “well-behaved” numbers is not just closed under multiplication — every element can also be *undone*.

A quick note on the word “group.” Mathematicians have a name for exactly this kind of structure: a set with an operation (here, multiplication mod n) that (i) is closed (combining two elements of the set stays in the set), (ii) has an identity element (here, 1, which does nothing when multiplied), and (iii) gives every element an inverse within the set. Such a structure is called a **group**. You do not need to know any general group theory to follow this lecture — we will only ever use the three properties just described, applied to this one specific set. We just want a name for it, because we will refer to it constantly:

$$(\mathbb{Z}/n\mathbb{Z})^* = \{a \in \{1, \dots, n-1\} : \gcd(a, n) = 1\},$$

the set of “well-behaved,” invertible residues mod n . By definition, this set has exactly $\varphi(n)$ elements. **Euler’s theorem, stated next, is a fact about exactly this set.**

Theorem 1.1.6 (Euler’s Theorem). *If $\gcd(a, n) = 1$, then*

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Plain-English statement, before the proof. Take any a sharing no factor with n — i.e. any a in our “well-behaved” club $(\mathbb{Z}/n\mathbb{Z})^*$ — and start computing $a^1, a^2, a^3, \dots$ modulo n . Euler’s theorem guarantees that by the time you reach exponent $\varphi(n)$, you have landed back exactly on 1 — regardless of which coprime a you started with, and regardless of how large or small n is. This “guaranteed return to 1 after exactly $\varphi(n)$ steps” is the single fact RSA is built on: it is precisely what lets a carefully chosen *second* exponentiation undo the effect of a first one, which was the problem we set out to solve at the very start of this section.

Proof — the constructive version, worth doing by hand once. We prove it by directly multiplying every element of the group $(\mathbb{Z}/n\mathbb{Z})^* = \{g_1, \dots, g_{\varphi(n)}\}$ by a and watching what happens, rather than citing an abstract theorem.

Step 1: multiplying by a just reorders the group. Consider $ag_1, ag_2, \dots, ag_{\varphi(n)} \pmod{n}$. Each ag_i is still coprime to n (a product of two things coprime to n , by the closure property discussed above), so every value in this new list is again some element of $(\mathbb{Z}/n\mathbb{Z})^*$. No two of them coincide either: if $ag_i \equiv ag_j \pmod{n}$, then since $\gcd(a, n) = 1$ we may multiply both sides by $a^{-1} \pmod{n}$ (which exists precisely because $\gcd(a, n) = 1$) to get $g_i \equiv g_j \pmod{n}$, forcing $i = j$. So the list $ag_1, \dots, ag_{\varphi(n)}$ consists of $\varphi(n)$ *distinct* elements, all drawn from a set that only has $\varphi(n)$ elements total — which means it must simply be $\{g_1, \dots, g_{\varphi(n)}\}$, written out in a different order.

Step 2: multiply everything together and cancel. Both lists contain exactly the same elements (just reordered), so their products mod n agree:

$$(ag_1)(ag_2)\cdots(ag_{\varphi(n)}) \equiv g_1g_2\cdots g_{\varphi(n)} \pmod{n}.$$

The left side equals $a^{\varphi(n)} \cdot P$, where $P = g_1g_2\cdots g_{\varphi(n)}$. Since P is a product of things coprime to n , $\gcd(P, n) = 1$, so P has an inverse mod n . The equation $a^{\varphi(n)} \cdot P \equiv P \pmod{n}$ then gives, after multiplying both sides by $P^{-1} \pmod{n}$,

$$a^{\varphi(n)} \equiv 1 \pmod{n}. \quad \square$$

Remark 1.1.7 (The one-line abstract version, for readers who already know group theory). *The argument above is a proof of Lagrange's theorem specialized to this one group: $(\mathbb{Z}/n\mathbb{Z})^*$ is a finite group of order $\varphi(n)$, and for any element a of any finite group G , $a^{|G|}$ equals the identity. The longer proof above is the same idea, unpacked into elementary steps that use only “multiplying reorders a finite set” and “cancel a common invertible factor.”*

Example 1.1.8 (Euler's theorem, traced by hand). *Let $n = 10$, so $(\mathbb{Z}/10\mathbb{Z})^* = \{1, 3, 7, 9\}$ and $\varphi(10) = 4$. Take $a = 3$ and multiply every group element by 3, mod 10:*

$$3 \cdot 1 = 3, \quad 3 \cdot 3 = 9, \quad 3 \cdot 7 = 21 \equiv 1, \quad 3 \cdot 9 = 27 \equiv 7.$$

The results $\{3, 9, 1, 7\}$ are exactly $\{1, 3, 7, 9\}$ again, just shuffled — precisely Step 1 of the proof. Multiplying the original four elements together: $P = 1 \cdot 3 \cdot 7 \cdot 9 = 189$. The proof says $3^4 \cdot P \equiv P \pmod{10}$, i.e. $3^4 \equiv 1 \pmod{10}$ after cancelling P . Direct check: $3^4 = 81 = 8 \cdot 10 + 1 \equiv 1 \pmod{10}$. ✓

Example 1.1.9 (Euler's theorem in action). *Let $a = 7$, $n = 10$. Since $\gcd(7, 10) = 1$ and $\varphi(10) = 4$, Euler's theorem predicts $7^4 \equiv 1 \pmod{10}$. Check: $7^2 = 49 \equiv 9$, $7^4 \equiv 9^2 = 81 \equiv 1 \pmod{10}$. ✓*

Corollary 1.1.10 (Fermat's Little Theorem). *If p is prime and $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$. This is Euler's theorem with $n = p$, since $\varphi(p) = p - 1$ (every integer from 1 to $p - 1$ is automatically coprime to a prime p).*

Back to the actual problem: how does this make decryption undo encryption?

Recall the goal from the very start of this section: we need $m^{ed} \equiv m \pmod{n}$ to hold, for suitably chosen e and d . We now have exactly the tool needed. Suppose e and d are chosen so that

$$ed \equiv 1 \pmod{\varphi(n)}.$$

Read this condition in words first: it says ed , divided by $\varphi(n)$, leaves remainder 1 — equivalently, ed is *one more than some whole number of multiples of $\varphi(n)$* . Call that whole number of multiples k , so as an equation over the integers (not just modulo $\varphi(n)$):

$$ed = 1 + k\varphi(n), \quad \text{for some integer } k \geq 0.$$

Substitute this into a^{ed} and simplify one step at a time, assuming for now $\gcd(a, n) = 1$:

$$a^{ed} = a^{1+k\varphi(n)} = a^1 \cdot a^{k\varphi(n)} = a \cdot \left(a^{\varphi(n)}\right)^k.$$

The first equality just substitutes $ed = 1 + k\varphi(n)$; the second splits the exponent's sum into a product of powers (the exponent law $x^{m+n} = x^m x^n$); the third regroups $a^{k\varphi(n)} = (a^{\varphi(n)})^k$

(the exponent law $x^{mn} = (x^m)^n$). Now Euler’s theorem enters: since $\gcd(a, n) = 1$, $a^{\varphi(n)} \equiv 1 \pmod{n}$. Substituting,

$$a \cdot \left(a^{\varphi(n)}\right)^k \equiv a \cdot 1^k \equiv a \pmod{n}.$$

Chaining the two displayed calculations together gives exactly $a^{ed} \equiv a \pmod{n}$ — decryption undoes encryption, which is exactly the problem we set out to solve.

Remark 1.1.11 (The “clock” intuition for the same argument, without the algebra). *Picture the powers of a modulo n arranged around a dial with $\varphi(n)$ positions. Euler’s theorem says raising a to the power $\varphi(n)$ is “one full lap” of the dial: you end up exactly back where you started, at 1. Raising a to the power $k\varphi(n)$ is “ k full laps” — still exactly back at 1, however many laps. Raising a to the power $ed = 1 + k\varphi(n)$ is therefore “ k full laps, plus one extra step” — and one extra step past “back where you started” is just $a^1 = a$. Choosing e and d with $ed \equiv 1 \pmod{\varphi(n)}$ is precisely choosing them so encrypting-then-decrypting is always some whole number of full laps plus exactly one step.*

Example 1.1.12 (The exponent identity, verified with actual numbers). *Take $n = 10$, so $\varphi(n) = 4$, and pick $e = 3$. We need d with $3d \equiv 1 \pmod{4}$; $d = 3$ works, since $3 \cdot 3 = 9 = 1 + 2 \cdot 4$ (so $k = 2$ here). Take $a = 7$ (coprime to 10). Encrypt-then-decrypt: $a^e = 7^3 = 343 \equiv 3 \pmod{10}$, then $(a^e)^d = 3^3 = 27 \equiv 7 \pmod{10}$ — back to the original $a = 7$, exactly as the identity above guarantees. Note $ed = 9 = 1 + 2 \cdot 4$: two full laps around a 4-position dial, plus one extra step.*

This is *exactly* the decryption-undoes-encryption identity RSA needs, and Section 1.2 turns it into an actual key-generation recipe: pick e first, then solve $ed \equiv 1 \pmod{\varphi(n)}$ for d using the Extended Euclidean Algorithm (precisely how one computes a multiplicative inverse — here, the inverse of e modulo $\varphi(n)$). There is one loose thread left, and it’s worth being upfront about it rather than sweeping it under the rug: the whole derivation above assumed $\gcd(a, n) = 1$. A real message a sent through RSA might, in principle, share a factor with n — and when it does, Euler’s theorem as stated simply does not apply to it, since Euler’s theorem is only a statement about the “well-behaved club” $(\mathbb{Z}/n\mathbb{Z})^*$. Section 1.3 closes this gap completely and turns our derivation above into a proof that holds for *every* message, and it is exactly where the Chinese Remainder Theorem — introduced next — does its first job for us.

1.1.3 The Chinese Remainder Theorem

Why we need this at all. We just flagged the gap: Euler’s theorem only talks about messages coprime to n . CRT is the tool that closes that gap for every message, and — as a bonus, unrelated to correctness — it will also give us a $4\times$ decryption speedup later (Section 1.2.3). Before touching the formal statement, let’s understand what CRT *says*, with actual numbers.

Step 1: the basic idea, with a tiny puzzle

Suppose you’re told a number x leaves remainder 2 when divided by 3, and remainder 3 when divided by 5. In symbols,

$$x \equiv 2 \pmod{3}, \quad x \equiv 3 \pmod{5}.$$

Is there such a number? Is it the only one? CRT exists to answer exactly this kind of question.

Step 2: solve it by brute force first

Before any formula, just think it through. Numbers leaving remainder 2 when divided by 3 are

$$2, 5, 8, 11, 14, 17, 20, \dots$$

Now check each one against the second condition (remainder 3 when divided by 5):

$$2 \div 5 \rightarrow \text{rem } 2 \ (\times), \quad 5 \div 5 \rightarrow \text{rem } 0 \ (\times), \quad 8 \div 5 \rightarrow \text{rem } 3 \ (\checkmark).$$

So $x = 8$ works. And so does 23, and 38, and 53 — every number 15 apart from 8, because $3 \times 5 = 15$. So the full answer isn't just " $x = 8$," it's

$$x \equiv 8 \pmod{15}.$$

That's the pattern CRT formalizes: two remainder conditions, on moduli sharing no common factor, pin down x uniquely *once you fix the modulus* $15 = 3 \times 5$ — not a single number, but a single residue class mod 15.

Step 3: the general statement, in words

If you know x 's remainder mod p and x 's remainder mod q , where p and q share no common factor, that is *exactly as much information* as knowing x 's remainder mod pq directly. Nothing is lost by splitting the modulus, and nothing is ambiguous: every pair of remainders corresponds to one and only one $x \bmod pq$. This is the fact RSA is built on — it lets us trade one hard computation mod $n = pq$ for two easier computations, mod p and mod q separately, with a guarantee that stitching the two answers back together recovers the truth mod n .

Remark 1.1.13 (A mental picture: two interlocking gears). *Picture two gears with p and q teeth, both starting at a marked tooth and turning together. If p and q share no common factor, the combined picture — “which tooth of gear 1 is marked now, which tooth of gear 2 is marked now” — does not repeat until pq turns have passed, and every one of the pq possible combinations occurs exactly once per cycle. “Remainder mod p ” and “remainder mod q ” are the two gears' positions; CRT says these two positions, taken together, pin down the combined state uniquely — the same information as one remainder mod pq .*

Step 4: the formal theorem

Theorem 1.1.14 (CRT). *Let $n = pq$ with p, q coprime. The map*

$$\mathbb{Z}/n\mathbb{Z} \longrightarrow \mathbb{Z}/p\mathbb{Z} \times \mathbb{Z}/q\mathbb{Z}, \quad x \mapsto (x \bmod p, x \bmod q)$$

is a ring isomorphism. Equivalently: the system

$$x \equiv a \pmod{p}, \quad x \equiv b \pmod{q}$$

has a unique solution $x \in \{0, \dots, n-1\}$ for every pair (a, b) .

Unpacking “ring isomorphism” one word at a time. “Map” just means: send x to the pair of its two remainders — exactly what we did by hand in Step 2. “Isomorphism” means this map is a perfect one-to-one correspondence between $\{0, \dots, n-1\}$ and all pairs (a, b) — this is the “unique solution” half of the theorem, and it's the part the puzzle in Step 1–2 already illustrated concretely. “Ring” (rather than just “set”) adds one more guarantee: the correspondence also respects *arithmetic*. Adding (or multiplying) two numbers mod n and then

splitting into remainders gives the same answer as splitting first and adding (or multiplying) each coordinate separately:

$$x+y \mapsto (a+c \bmod p, b+d \bmod q), \quad xy \mapsto (ac \bmod p, bd \bmod q) \quad \text{when } x \mapsto (a, b), y \mapsto (c, d).$$

This extra guarantee is what makes CRT more than a counting trick: an entire arithmetic computation can be carried out *coordinatewise*, mod p and mod q separately, and the two partial answers automatically recombine correctly mod n — exactly the mechanism Section 1.2.3 exploits for faster decryption, and Section 1.3 exploits for a correctness proof covering every message, not just the coprime ones.

Proof, explained step by step. We check three things: the map preserves arithmetic, it never sends two different numbers to the same pair, and — by a counting argument — it must therefore hit every pair.

It preserves arithmetic. If $x \equiv a \pmod{p}$ and $y \equiv c \pmod{p}$, then directly from the definition of congruence, $x+y \equiv a+c \pmod{p}$ and $xy \equiv ac \pmod{p}$ — and likewise mod q . So reducing mod p (or mod q) *after* adding/multiplying gives the same result as adding/multiplying the already-reduced remainders. That’s exactly the coordinatewise formula above.

It is injective (no two different numbers share a remainder pair). Suppose x and y , both in $\{0, \dots, n-1\}$, map to the *same* pair: $x \equiv y \pmod{p}$ and $x \equiv y \pmod{q}$. The first says $p \mid (x-y)$; the second says $q \mid (x-y)$. Since p, q are distinct primes, a number divisible by both must be divisible by their product: $pq \mid (x-y)$. But $|x-y| < n = pq$, and the only multiple of pq smaller than pq in absolute value is 0 — so $x = y$. Distinct inputs always land on distinct output pairs.

It is therefore also surjective (every pair is hit). The domain $\{0, \dots, n-1\}$ has exactly $n = pq$ elements. The codomain — all pairs (a, b) with $0 \leq a < p$, $0 \leq b < q$ — also has exactly $p \cdot q = n$ elements. An injective map between two finite sets of the *same size* must also be surjective: if some pair were missed, the n distinct images would have to squeeze into fewer than n remaining slots, which is impossible. So every pair (a, b) is the image of exactly one x — the “unique solution” claim. $\square$

Step 5: not just existence — a formula that actually builds x

The proof above guarantees a solution exists, but doesn’t hand us one. Here is a constructive recipe, using the Extended Euclidean Algorithm to compute an inverse:

1. Compute $q^{-1} \bmod p$ (this exists because $\gcd(p, q) = 1$).
2. The unique solution in $\{0, \dots, n-1\}$ to $x \equiv a \pmod{p}$, $x \equiv b \pmod{q}$ is

$$x = b + q \cdot \left(((a-b) \cdot (q^{-1} \bmod p)) \bmod p \right).$$

Read this informally: start from $x = b$, which already satisfies the mod- q condition (since $b < q$). Then add a correction term that is a multiple of q — chosen so the result *also* satisfies the mod- p condition — without disturbing the mod- q condition, because adding any multiple of q never changes a value’s remainder mod q .

Example 1.1.15 (Solving the Step-1 puzzle two ways). *Solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$ — so $p = 3$, $a = 2$, $q = 5$, $b = 3$, $n = 15$, the exact puzzle from Step 1.*

By listing (*Step 2 above, fine for tiny numbers but doesn’t scale*): we already found $x = 8$, i.e. $x \equiv 8 \pmod{15}$.

By the formula (the method that scales, and the one RSA actually uses): first compute $q^{-1} \bmod p = 5^{-1} \bmod 3$. Since $5 \equiv 2 \pmod{3}$ and $2 \cdot 2 = 4 \equiv 1 \pmod{3}$, we get $5^{-1} \equiv 2 \pmod{3}$. Then

$$x = b + q \cdot \left(((a - b) \cdot (q^{-1} \bmod p)) \bmod p \right) = 3 + 5 \cdot (((2 - 3) \cdot 2) \bmod 3) = 3 + 5 \cdot ((-2) \bmod 3) = 3 + 5 \cdot 1 = 8.$$

Both methods agree: $x = 8$. This same computation — reduce mod p , reduce mod q , recombine with a modular-inverse correction term — is exactly what Section 1.2.3's CRT-RSA decryption algorithm performs, with a, b replaced by the two partial decryptions m_p, m_q .

Remark 1.1.16 (Beyond two moduli: the general formula, if you've seen it elsewhere). Everything above uses exactly two moduli (p and q), since that's all RSA needs. But CRT works for any number of pairwise-coprime moduli $n_1, \dots, n_k$, and it's worth seeing the general pattern once so the two-modulus case above looks like a special case rather than a different theorem. Given

$$x \equiv a_1 \pmod{n_1}, \quad x \equiv a_2 \pmod{n_2}, \quad \dots, \quad x \equiv a_k \pmod{n_k},$$

set $N = n_1 n_2 \cdots n_k$, and for each i let $N_i = N/n_i$ (the product of all moduli except n_i) and let M_i be the inverse of $N_i \bmod n_i$, i.e. $N_i M_i \equiv 1 \pmod{n_i}$. Then

$$x \equiv \sum_{i=1}^k a_i N_i M_i \pmod{N}.$$

Why this works: each term $a_j N_j M_j$ is built so that mod n_i it vanishes for every $j \neq i$ (because N_j is a multiple of n_i whenever $j \neq i$) and reduces to exactly a_i when $j = i$ (because $N_i M_i \equiv 1 \pmod{n_i}$ by construction) — so the whole sum matches $a_i \bmod n_i$, for every i simultaneously. Re-deriving the Step-1 puzzle this way: $N = 15$, $N_1 = 5$, $N_2 = 3$; $M_1 = 5^{-1} \bmod 3 = 2$ and $M_2 = 3^{-1} \bmod 5 = 2$; so $x \equiv 2 \cdot 5 \cdot 2 + 3 \cdot 3 \cdot 2 = 20 + 18 = 38 \equiv 8 \pmod{15}$ — the same $x = 8$ as before. (The two-modulus formula in Step 5 is just this general formula specialized to $k = 2$, rearranged into a slightly more convenient form.)

Remark 1.1.17 (Two uses of CRT in this lecture — and why they are genuinely different jobs). CRT plays two completely different roles in RSA that students often conflate, precisely because both roles use the exact same theorem:

1. **Correctness** (Section 1.3): Euler's theorem alone only talks about a with $\gcd(a, n) = 1$. CRT lets us handle every message $a \in \{0, \dots, n - 1\}$, including the astronomically rare ones sharing a factor with n , by checking correctness separately mod p and mod q (where a simpler, case-by-case argument works) and using CRT to stitch the two into one statement mod n .
2. **Efficiency** (Section 1.2.3): the private-key holder, who knows p and q , replaces one expensive exponentiation mod the full-size n with two cheaper exponentiations mod the half-size p and q , then uses exactly the reconstruction formula above to recombine the two partial results into the true answer mod n . This is roughly $4\times$ faster — and, as a preview, is precisely the optimization the Bellcore fault attack (Lesson 3) exploits: corrupting just one of the two halves and recombining anyway produces a faulty signature that leaks a whole prime factor of n .

1.2 RSA Construction

We now have everything we need. The refresher section built exactly one tool: a way to choose e and d so that $m^{ed} \equiv m \pmod{n}$ — i.e. so that “encrypt then decrypt” is the identity operation. Key generation below is nothing more than setting up the numbers so that tool applies; encryption and decryption are nothing more than *using* it.

1.2.1 Key Generation

Algorithm 1 RSA Key Generation

- 1: Choose two large distinct primes p, q of similar bit-length (typically 1536 bits each for a 3072-bit modulus)
 - 2: Compute $n = pq$
 - 3: Compute $\varphi(n) = (p-1)(q-1)$
 - 4: Choose e with $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$
 - 5: Compute $d \equiv e^{-1} \pmod{\varphi(n)}$ via the Extended Euclidean Algorithm
 - 6: **Public key:** (n, e) **Private key:** (n, d) (or (p, q, d) for CRT speedup)
-

Definition 1.2.1 (RSA Encryption / Decryption). *For a message $m \in \{0, 1, \dots, n-1\}$:*

$$\text{Encryption: } c = m^e \bmod n, \quad \text{Decryption: } m = c^d \bmod n.$$

Remark 1.2.2 (Why $\gcd(e, \varphi(n)) = 1$ is required — and why this is a completely different condition from $\gcd(a, n) = 1$). *It is easy to conflate two superficially similar-looking coprimality conditions in this lecture, so it is worth pulling them apart explicitly, side by side:*

	<i>messages / bases a</i>	<i>exponents e, d</i>
<i>live in</i>	$\{0, \dots, n-1\}$	$\{0, \dots, \varphi(n)-1\}$
<i>“nice” subset</i>	a with $\gcd(a, n) = 1$	e with $\gcd(e, \varphi(n)) = 1$
<i>why</i>	lets Euler’s theorem apply to a	lets e be inverted mod $\varphi(n)$

These are the same shape of condition (“coprime to the modulus you’re working in”), but applied to two genuinely different moduli, one level apart: bases are filtered against n , exponents are filtered against $\varphi(n)$. Do not read “ $1 < e < \varphi(n)$ ” as merely naming a range to pick a number from — the range itself is forced (exponents provably repeat with period $\varphi(n)$, exactly as bases repeat with period n ; see the paragraph below), but landing inside that range is not sufficient on its own: only the $\varphi(\varphi(n))$ values in that range that are coprime to $\varphi(n)$ are actually usable as an encryption exponent.

Why the range is $\{0, \dots, \varphi(n)-1\}$ at all. *This is a separate fact from the coprimality filter, and it is worth seeing why exponents wrap around with period $\varphi(n)$, not n . For any a coprime to n and any integer x , Euler’s theorem gives*

$$a^{x+\varphi(n)} = a^x \cdot a^{\varphi(n)} \equiv a^x \cdot 1 = a^x \pmod{n},$$

so raising a to the power e and to the power $e + \varphi(n)$ produces the identical ciphertext, every time. Exponents, for the purposes of RSA, genuinely live in $\mathbb{Z}/\varphi(n)\mathbb{Z}$ — a clock with $\varphi(n)$ positions — exactly parallel to how bases live in $\mathbb{Z}/n\mathbb{Z}$, a clock with n positions. Picking e larger than $\varphi(n)$ is not wrong, merely wasteful: it computes the exact same encryption function as $e \bmod \varphi(n)$, using more squarings to get there.

Why coprimality to $\varphi(n)$, specifically, is needed on top of that. *We need e to be invertible modulo $\varphi(n)$, i.e. we need some d with $ed \equiv 1 \pmod{\varphi(n)}$. An element of $\mathbb{Z}/\varphi(n)\mathbb{Z}$ is invertible if and only if it is coprime to $\varphi(n)$ (the same standard fact used throughout this lecture: $(\mathbb{Z}/k\mathbb{Z})^*$ consists exactly of residues coprime to k — here applied with $k = \varphi(n)$ rather than $k = n$). Without this, no valid d exists at all, and encryption under such an e could never be undone.*

Example 1.2.3 (The filter, made fully concrete on $n = 10$). Take $n = 10$, so $\varphi(10) = 4$ and exponents reduce modulo 4: the only candidates are $e \in \{0, 1, 2, 3\}$. Check each one directly against $\gcd(e, 4)$:

e	0	1	2	3
$\gcd(e, 4)$	4	1	2	1

$e = 0$ is useless regardless ($m^0 = 1$ always, and it isn't invertible: $\gcd(0, 4) = 4 \neq 1$). $e = 1$ is invertible ($d = 1$) but trivial ("encryption" that does nothing). $e = 2$ has $\gcd(2, 4) = 2 \neq 1$ — **no d exists**: checking every value of $d \bmod 4$ directly, $2 \cdot 0 \equiv 0$, $2 \cdot 1 \equiv 2$, $2 \cdot 2 \equiv 0$, $2 \cdot 3 \equiv 2 \pmod{4}$, and 1 never appears, so $e = 2$ can never be decrypted and must be excluded. $e = 3$ has $\gcd(3, 4) = 1$, and indeed $3 \cdot 3 = 9 \equiv 1 \pmod{4}$, giving $d = 3$ — this is the only usable, nontrivial exponent for $n = 10$, and it is exactly the pair used in the worked examples of Section 1.1.3 and earlier in this section.

Notice the parallel exactly: just as the "nice" bases mod 10 are the coprime-to-10 subset $\{1, 3, 7, 9\} \subset \{0, \dots, 9\}$, the "nice" exponents mod $\varphi(10) = 4$ are the coprime-to-4 subset $\{1, 3\} \subset \{0, \dots, 3\}$ — the same filtering idea, one level up, applied to $\varphi(n)$ instead of n .

At realistic RSA sizes, $\varphi(n)$ has hundreds of digits, so this filter is never applied by listing candidates (as in the tiny example above) — it is applied by picking a candidate e (in practice, almost always the fixed value 65537) and directly testing $\gcd(e, \varphi(n)) = 1$ via the Euclidean algorithm, exactly as Section 1.5.2 discusses.

Example 1.2.4 (Toy example — never use these sizes in practice). Let $p = 61$, $q = 53$. Then $n = 3233$, $\varphi(n) = 60 \cdot 52 = 3120$. Choose $e = 17$ (check $\gcd(17, 3120) = 1$). Extended Euclid gives $d = 2753$, since $17 \times 2753 = 46801 = 15 \times 3120 + 1$. Encrypt $m = 65$: $c = 65^{17} \bmod 3233 = 2790$. Decrypt: $2790^{2753} \bmod 3233 = 65$. ✓

1.2.2 Fast Modular Exponentiation

Both encryption and decryption require computing $a^k \bmod n$ efficiently. Naive repeated multiplication is $O(k)$ multiplications; **square-and-multiply** reduces this to $O(\log k)$.

Algorithm 2 Square-and-Multiply (Right-to-Left)

```

1: function MODEXP( $a, k, n$ )
2:    $result \leftarrow 1$ 
3:    $base \leftarrow a \bmod n$ 
4:   while  $k > 0$  do
5:     if  $k \bmod 2 = 1$  then
6:        $result \leftarrow (result \times base) \bmod n$ 
7:     end if
8:      $base \leftarrow (base \times base) \bmod n$ 
9:      $k \leftarrow \lfloor k/2 \rfloor$ 
10:  end while
11:  return  $result$ 
12: end function

```

Example 1.2.5 (Square-and-multiply trace). Compute $5^{117} \bmod 19$. Write $117 = 1110101_2$

(bits, LSB first: 1, 0, 1, 0, 1, 1, 1). Trace of MODEXP, showing (bit consumed, result, base after squaring):

step	bit	result	base
1	1	5	6
2	0	5	17
3	1	9	4
4	0	9	16
5	1	11	9
6	1	4	5
7	1	1	6

Final result: 1. Direct check: $5^{117} \bmod 19 = 1$ (consistent, since $\text{ord}_{19}(5) \mid \varphi(19) = 18$, and indeed $117 \equiv 9 \pmod{18}$ while $5^9 \equiv 1 \pmod{19}$ — try verifying this independently). Only **7 squarings and 5 multiplications** were needed instead of 116 successive multiplications.

Remark 1.2.6 (Security-relevant note — preview of Lesson 3). The naive implementation above has data-dependent branching (the *if* depends on the secret exponent's bits), which leaks timing information. This is the root cause of timing attacks on RSA, covered when we do implementation attacks.

1.2.3 CRT-Based Decryption Speedup

Since the private-key holder knows p and q , decryption can be done mod p and mod q separately (smaller exponents, smaller moduli) and recombined via CRT.

Algorithm 3 CRT-RSA Decryption

- 1: Precompute: $d_p = d \bmod (p-1)$, $d_q = d \bmod (q-1)$, $q_{inv} = q^{-1} \bmod p$
 - 2: $m_p = c^{d_p} \bmod p$
 - 3: $m_q = c^{d_q} \bmod q$
 - 4: $h = q_{inv} \cdot (m_p - m_q) \bmod p$
 - 5: $m = m_q + h \cdot q$
 - 6: **return** m
-

Example 1.2.7 (CRT-RSA decryption, continuing the toy example). Recall $p = 61, q = 53, n = 3233, e = 17, d = 2753$, and $c = 2790$ (encryption of $m = 65$ from the earlier example). Precompute:

$$d_p = d \bmod (p-1) = 2753 \bmod 60 = 53, \quad d_q = d \bmod (q-1) = 2753 \bmod 52 = 49, \quad q_{inv} = q^{-1} \bmod p = 38$$

Then:

$$m_p = c^{d_p} \bmod p = 2790^{53} \bmod 61 = 4, \quad m_q = c^{d_q} \bmod q = 2790^{49} \bmod 53 = 12.$$

Recombine: $h = q_{inv}(m_p - m_q) \bmod p = 38 \cdot (4 - 12) \bmod 61 = 1$, and $m = m_q + h \cdot q = 12 + 1 \cdot 53 = 65$. ✓ Matches the direct decryption, but every intermediate exponentiation used a ~ 6 -bit exponent and a ~ 6 -bit modulus instead of the full-size d and n .

This gives roughly a $4\times$ speedup (two exponentiations with half-size exponents and half-size moduli, each individually $\sim 8\times$ cheaper, combined with cheap recombination). **Flag this in your memory:** this optimization is exactly what the Bellcore fault attack (Lesson 3) exploits — a single computational fault in either the m_p or m_q branch lets an attacker factor n from one faulty signature.

1.3 Correctness of RSA

Recall the loose thread we flagged back in Section 1.1.3: our derivation of $m^{ed} \equiv m \pmod{n}$ assumed $\gcd(m, n) = 1$. We now close that gap and prove $\text{Dec}(\text{Enc}(m)) = m$ for *every* $m \in \{0, \dots, n-1\}$ — not just those coprime to n . Most textbooks quietly assume $\gcd(m, n) = 1$ and invoke Euler’s theorem directly; a rigorous treatment must handle the small-probability edge case, and this is exactly where CRT (Theorem 1.1.14) earns its keep: the strategy is to prove the congruence *separately* modulo p and modulo q (where a short case split handles the edge case cleanly) and then let CRT stitch the two into a single statement modulo $n = pq$.

Theorem 1.3.1 (RSA Correctness). *For all $m \in \{0, \dots, n-1\}$, $m^{ed} \equiv m \pmod{n}$.*

Proof. Since $ed \equiv 1 \pmod{\varphi(n)}$, write $ed = 1 + k\varphi(n) = 1 + k(p-1)(q-1)$ for some integer $k \geq 0$.

By CRT (Theorem 1.1.14), it suffices to show

$$m^{ed} \equiv m \pmod{p} \quad \text{and} \quad m^{ed} \equiv m \pmod{q},$$

since these together are equivalent to $m^{ed} \equiv m \pmod{pq}$.

We show the first congruence; the second is symmetric. Consider two cases.

Case 1: $p \nmid m$. Then $\gcd(m, p) = 1$, so by Fermat’s Little Theorem $m^{p-1} \equiv 1 \pmod{p}$. Hence

$$m^{ed} = m^{1+k(p-1)(q-1)} = m \cdot (m^{p-1})^{k(q-1)} \equiv m \cdot 1^{k(q-1)} \equiv m \pmod{p}.$$

Case 2: $p \mid m$. Then $m \equiv 0 \pmod{p}$, and since $ed \geq 1$, $m^{ed} \equiv 0 \equiv m \pmod{p}$ trivially.

Either way, $m^{ed} \equiv m \pmod{p}$. By the symmetric argument, $m^{ed} \equiv m \pmod{q}$. By CRT, $m^{ed} \equiv m \pmod{pq} = \pmod{n}$. $\square$

Remark 1.3.2 (Why this edge case matters pedagogically, not just practically). *The event $p \mid m$ for a random $m \in \{0, \dots, n-1\}$ has probability $1/p$ — utterly negligible for cryptographic-size primes, so it is never a practical concern. But it matters conceptually: it shows correctness of RSA is really a CRT argument dressed up, and it is the reason the correctness proof cannot be “just Euler’s theorem” as often claimed. This distinction resurfaces when we discuss why RSA’s algebraic structure (a ring homomorphism argument, not merely a group-order argument) enables attacks like Håstad’s broadcast attack in Lesson 2.*

Example 1.3.3 (The edge case, made concrete). *With $n = 3233 = 61 \times 53$, the “bad” messages are the tiny fraction that are multiples of 61 or 53: e.g., $m = 61$. Here $p \mid m$, so Case 2 applies mod 61 ($m^{ed} \equiv 0 \equiv m \pmod{61}$ trivially), while Case 1 applies mod 53 (since $\gcd(61, 53) = 1$, Fermat’s little theorem does the work mod 53). CRT stitches the two together. You can verify numerically that $61^{17 \cdot 2753} \pmod{3233} = 61$ still holds — correctness survives even though Euler’s theorem alone (which needs $\gcd(m, n) = 1$) does not directly apply to $m = 61$, since $\gcd(61, 3233) = 61 \neq 1$.*

Exercise 1.3.4. *Prove the case $q \mid m$ directly (mirror the argument above), and explain why $p \mid m$ and $q \mid m$ cannot both hold when $m < n = pq$ and $m \neq 0$.*

1.4 Security Foundations

We have now shown RSA is *correct* — decryption always recovers the message. That says nothing yet about whether RSA is *secure* — whether an eavesdropper who sees only the public key and a ciphertext can recover the message. This section pins down precisely what “secure” means for RSA and how it relates to the more familiar problem of factoring n .

1.4.1 The RSA Problem and the RSA Assumption

Definition 1.4.1 (RSA Problem). *Given (n, e) and $c = m^e \bmod n$ for a uniformly random $m \in \mathbb{Z}_n^*$, compute m .*

Definition 1.4.2 (RSA Assumption). *No probabilistic polynomial-time algorithm solves the RSA problem with non-negligible probability (over the choice of p, q, e, m), for suitably generated n .*

The critical logical relationship, stated first in plain terms. One direction is completely obvious once you say it out loud: if you can factor $n = pq$, you immediately recover $\varphi(n) = (p-1)(q-1)$ (Proposition 1.1.3), then compute $d = e^{-1} \bmod \varphi(n)$ exactly as in key generation, and now you can decrypt anything — you have effectively rebuilt the private key from scratch. So:

$$\text{Factoring } n \implies \text{Breaking RSA.}$$

In the notation of reductions, **Factoring** $\leq_p$ **Breaking RSA**: factoring reduces to breaking RSA, meaning an efficient factoring algorithm immediately gives an efficient RSA-breaking algorithm. What is *not* obvious, and turns out not to be known, is the reverse question.

The open question, since 1977, is whether the reduction goes the other way: does an oracle that breaks RSA (computes e -th roots mod n) let you factor n ? This is **not known in general**. There exist special/weakened settings (e.g., Rabin’s cryptosystem, and RSA with a specific low-exponent oracle in certain models) where equivalence to factoring *can* be shown, but for standard RSA the assumption is a genuinely separate, slightly stronger-looking, hardness assumption than “factoring is hard.”

Remark 1.4.3 (Why this matters for how we teach attacks). *Every specific attack technique in Lessons 2–3 (Wiener, Coppersmith, Håstad, Bleichenbacher, timing/fault attacks) breaks RSA without factoring n . This is the point: they exploit structural weaknesses in how RSA is used (small d , small e with related messages, malleable padding, faulty computation) rather than solving the underlying hard problem. Keep this distinction sharp for students: “RSA is broken” almost always means “this particular use of RSA is broken,” not “factoring got easy.”*

1.4.2 Why Textbook RSA Is Not Secure As-Is

Even setting aside implementation attacks, **textbook RSA** (encrypting $m^e \bmod n$ directly, with no padding) has structural problems worth flagging now, to motivate why Lesson 3 exists at all:

- **Deterministic**: the same plaintext always produces the same ciphertext under a given key $\Rightarrow$ no semantic security; an attacker can build a dictionary of likely plaintexts and their ciphertexts.
- **Malleable**: given $c = m^e \bmod n$, anyone can compute $c' = (2^e \cdot c) \bmod n = (2m)^e \bmod n$, a valid encryption of $2m$, without knowing m . No integrity guarantee whatsoever.

- **Small-message / small-exponent issues:** if $m^e < n$ (no modular reduction occurs), $c = m^e$ over the integers, and m is recovered by an ordinary integer e -th root — no cryptography needed. This is the seed of Håstad's broadcast attack (Lesson 2).

Example 1.4.4 (Malleability, worked). Using $n = 3233, e = 17, d = 2753$ and $c = 2790$ (encryption of $m = 65$): compute $c' = (2^{17} \bmod 3233) \cdot c \bmod 3233 = 3017$. Decrypting, $c'^d \bmod n = 130 = 2 \times 65$. An attacker who intercepts c and wants to double the plaintext (e.g., doubling a monetary amount in a naive protocol) needs no key material at all to do this.

Example 1.4.5 (No modular wraparound, worked). Let $p = 17, q = 19$, so $n = 323$, and suppose $e = 3$ (small exponent, no padding). Encrypt $m = 5$: $c = 5^3 = 125$. Since $125 < 323$, no reduction mod n ever occurred — c is just the integer 125. An eavesdropper who does not know d, p , or q recovers m by taking an ordinary integer cube root: $\sqrt[3]{125} = 5$. This is not a flaw in the mathematics of RSA — it is a flaw in using RSA without padding on small messages with small e , and it generalizes (via CRT, not integer roots) to Håstad's broadcast attack in Lesson 2, where the message need not literally satisfy $m^e < n$.

This is why real systems use **padded RSA** (PKCS#1 v1.5, OAEP) — covered in Lesson 3 alongside the attacks that padding is designed to prevent, and the attacks that emerged *against* specific padding schemes (Bleichenbacher).

1.5 Practical Key Generation

1.5.1 Primality Testing

Key generation needs to sample large primes. We cannot test primality by trial division for 1536-bit numbers, so we use **probabilistic primality tests**.

Miller–Rabin Primality Test (statement only — proof is number-theoretic and out of scope here). Write $n - 1 = 2^s d$ with d odd. Pick random witnesses $a \in \{2, \dots, n - 2\}$ and check whether

$$a^d \equiv 1 \pmod{n} \quad \text{or} \quad a^{2^r d} \equiv -1 \pmod{n} \text{ for some } 0 \leq r < s.$$

If neither holds, n is *composite* (certain). If it holds, n is *probably prime*: the test has false-positive probability $\leq 1/4$ per random witness, so k independent witnesses give error probability $\leq 4^{-k}$. In practice $k \approx 40$ is used for cryptographic key generation, giving error probability $< 2^{-80}$.

Example 1.5.1 (Miller–Rabin, worked — and why one witness isn’t enough). Let $n = 91 = 7 \times 13$ (*composite*). Write $n - 1 = 90 = 2^1 \cdot 45$, so $s = 1, d = 45$.

Witness $a = 3$: $3^{45} \bmod 91 = 27$. This is neither 1 nor $-1 \equiv 90 \pmod{91}$, and there are no further squarings to check ($s = 1$ means only $r = 0$ is available). So $a = 3$ correctly exposes 91 as composite.

Witness $a = 10$: $10^{45} \bmod 91 = 90 \equiv -1 \pmod{91}$. This passes the test — $a = 10$ is a false witness for compositeness here, illustrating exactly why the test is only probabilistic and why cryptographic key generation uses dozens of independent random witnesses rather than one.

1.5.2 Choosing e and d

- **Common choice:** $e = 65537 = 2^{16} + 1$. It is prime, has a Hamming weight of 2 (fast exponentiation: only 17 squarings + 1 multiplication), and is large enough to avoid the naive small- e pitfalls above.
- **Avoid $e = 3$** without proper padding — historically responsible for several real-world broadcast-style vulnerabilities (Lesson 2).
- d must be **large** — roughly the same bit-length as $\varphi(n)$. A small d (say $d < n^{0.25}$, roughly) is catastrophic: Wiener’s continued-fraction attack recovers it directly from (n, e) alone. Full derivation in Lesson 2.

1.5.3 Prime Generation Pitfalls

Naively generated primes can be individually “strong” (large, probably-prime) yet jointly **weak** if not generated with enough entropy or independence:

- **Shared-prime attacks:** if two different RSA moduli $n_1 = pq_1$ and $n_2 = pq_2$ share a prime factor p (due to poor entropy at key-generation time — this happened in real embedded devices), then $\gcd(n_1, n_2) = p$ instantly, computable in polynomial time via Euclid’s algorithm even across a database of millions of public keys. This was a real, large-scale finding (Heninger et al., 2012; Lenstra et al., 2012) affecting embedded/IoT devices with weak randomness.

Example 1.5.2 (Shared-prime attack, worked). Suppose (toy sizes) two devices generate $n_1 = p \cdot q_1$ and $n_2 = p \cdot q_2$ with a shared factor $p = 1000003$ due to a weak entropy pool at boot time, where $q_1 = 1009$, $q_2 = 2003$. Then $n_1 = 1009003027$ and $n_2 = 2003006009$. An

attacker with no other information computes $\gcd(n_1, n_2) = 1000003 = p$ via the Euclidean algorithm in microseconds, instantly factoring both moduli. At internet scale, this is done pairwise (or with a batch-gcd algorithm) across millions of collected public keys — no individual modulus looks weak in isolation, but shared factors across the population are what break.

- **Smooth $p-1$:** if $p-1$ has only small prime factors, p is vulnerable to Pollard’s $p-1$ method (Section 1.6.3). Historically “strong primes” (where $p-1$ has a large prime factor) were recommended as a defense; modern guidance considers this largely unnecessary if primes are simply chosen large and random, since the probability of accidentally-smooth $p-1$ is negligible at current key sizes — but it is worth knowing *why* the old advice existed.

1.6 Factoring Algorithms

We just saw that factoring n immediately breaks RSA (compute $\varphi(n)$, then d , then decrypt anything). So the entire security of RSA rests on factoring being *hard* — and “hard” is only meaningful relative to the best algorithm anyone actually knows. This section surveys those algorithms, from the hopeless (trial division) to the state of the art (the General Number Field Sieve), building the toolkit needed to justify — and periodically revise — key-size recommendations.

1.6.1 Trial Division and Fermat’s Method

Trial division: try all primes up to $\sqrt{n}$. Cost $O(\sqrt{n})$ — completely infeasible for cryptographic n (a 2048-bit n has $\sqrt{n} \approx 2^{1024}$).

Fermat’s Factorization Method. If $n = pq$ with p, q close together, write $n = a^2 - b^2 = (a - b)(a + b)$ where $a = \frac{p+q}{2}$, $b = \frac{p-q}{2}$. Search for a starting at $\lceil \sqrt{n} \rceil$, checking if $a^2 - n$ is a perfect square. If p, q differ by only a few bits, this converges almost immediately.

Example 1.6.1 (Fermat’s method, worked). *Let $p = 10007$, $q = 10009$ (deliberately close). Then $n = pq = 100160063$, and $\lceil \sqrt{n} \rceil = 10008$. Try $a = 10008$: $a^2 - n = 10008^2 - 100160063 = 1 = 1^2$, a perfect square with $b = 1$. Immediately, $p = a - b = 10007$ and $q = a + b = 10009$. One single trial recovered the full factorization — this is why close primes are catastrophic, regardless of how large n itself is.*

Practical consequence. Never choose p and q close to each other (e.g., both near $\sqrt{n}$ with only a few differing bits) — Fermat’s method factors such n almost instantly. This is why key-generation standards mandate p, q be independently random over their full bit-length range, not merely “two large primes.”

1.6.2 Pollard’s ρ Algorithm

A far better generic method, based on a birthday-paradox argument, with expected running time $O(n^{1/4})$ (i.e., $O(\sqrt{p})$ where p is the smallest prime factor).

Algorithm 4 Pollard’s ρ (Floyd cycle-detection variant)

```
1: Pick pseudo-random function  $f(x) = x^2 + c \bmod n$  (some small constant  $c \neq 0, -2$ )
2:  $x \leftarrow 2, y \leftarrow 2$ 
3: repeat
4:    $x \leftarrow f(x)$ 
5:    $y \leftarrow f(f(y))$ 
6:    $d \leftarrow \gcd(|x - y|, n)$ 
7: until  $d \neq 1$ 
8: if  $d = n$  then
9:   Restart with a different  $f$ 
10: else
11:   return  $d$  (a nontrivial factor)
12: end if
```

Heuristic argument for why this works. Treat f as a pseudo-random function on $\mathbb{Z}/p\mathbb{Z}$ (for p the unknown smallest factor). By the birthday paradox, the sequence $x_0, x_1, x_2, \dots \pmod{p}$ collides (enters a cycle) after roughly $O(\sqrt{p})$ steps, even though the same sequence mod n does not yet show any visible structure (since n is much larger). Once $x_i \equiv x_j \pmod{p}$ but $x_i \not\equiv x_j \pmod{n}$, we get $p \mid (x_i - x_j)$ but $n \nmid (x_i - x_j)$, so $\gcd(x_i - x_j, n)$ reveals p (or a multiple structure

containing it) without ever knowing p in advance. Floyd’s “tortoise and hare” technique detects this collision using $O(1)$ memory.

Example 1.6.2 (Pollard’s ρ , worked). *Let $n = 10403 = 101 \times 103$. Using $f(x) = x^2 + 1 \pmod n$, starting $x_0 = y_0 = 2$:*

<i>step</i>	<i>x</i>	<i>y</i>	$\gcd(x - y , n)$
1	5	26	1
2	26	598	1
3	677	3418	1
4	598	3531	1
5	3903	3724	1
6	3418	9812	1
7	156	9970	1
8	3531	3682	1
9	5168	7087	101

After just 9 iterations, a nontrivial factor 101 appears — versus 101 trial divisions (or $\sqrt{101} \approx 10$ under ρ ’s $O(\sqrt{p})$ bound, matching what we observe) to find the same factor by brute force on a number this small. The gap widens enormously as n grows.

Relevance to RSA: Pollard’s ρ is why RSA moduli need *both* prime factors to be large — it factors n in time proportional to $\sqrt{p_{\min}}$, the smallest factor, so an unbalanced $n = pq$ with a small p is broken quickly regardless of how large q is or how large n overall appears.

1.6.3 Pollard’s $p - 1$ Algorithm

Exploits primes p where $p - 1$ is **smooth** (product of only small primes).

Algorithm 5 Pollard’s $p - 1$

```

1: Choose smoothness bound  $B$ 
2:  $a \leftarrow 2$ 
3: for each prime  $\ell \leq B$  do
4:    $a \leftarrow a^{\ell^{\lfloor \log_{\ell} B \rfloor}} \pmod n$ 
5: end for
6:  $d \leftarrow \gcd(a - 1, n)$ 
7: if  $1 < d < n$  then
8:   return  $d$ 
9: end if
```

Why it works: if $p - 1$ divides $M = \text{lcm}(1, 2, \dots, B)$ (all its prime power factors are $\leq B$), then by Fermat’s little theorem $a^M \equiv 1 \pmod p$ for any a coprime to p , so $p \mid \gcd(a^M - 1, n)$, revealing p — while $q - 1$ (assumed non-smooth) does not similarly divide M , so $q \nmid (a^M - 1)$ typically, and the gcd isolates p cleanly.

Defense: choose p such that $p - 1$ has at least one large prime factor (“strong prime” criterion, historically recommended; largely moot at 2048+ bit sizes chosen with proper randomness, but essential to understand conceptually).

Example 1.6.3 (Pollard’s $p - 1$, worked). *Let $p = 1009$ (so $p - 1 = 1008 = 2^4 \cdot 3^2 \cdot 7$, smooth) and $q = 2027$ (so $q - 1 = 2026 = 2 \cdot 1013$, not smooth — 1013 is a large prime). Then $n = pq = 2045243$.*

Choose smoothness bound $B = 16$ (large enough that $2^4, 3^2, 7 \leq 16$ are all captured). Build $M = \prod_{\ell \leq B} \ell^{\lfloor \log_{\ell} 16 \rfloor} = 2^4 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11 \cdot 13 = 720720$, which is divisible by 1008. Compute $a = 2^M \pmod n = 2024055$, then $\gcd(a - 1, n) = \mathbf{1009} = p$. The attack succeeds because every

prime power dividing $p-1$ also divides M , so $2^M \equiv 1 \pmod{p}$ (Fermat), while $q-1 = 2 \cdot 1013$ does not divide M (since $1013 > B$), so no such collapse happens mod q — the gcd cleanly isolates p .

Contrast: attempting the same procedure with $B = 7$ (too small) fails — $\gcd(2^{M'} - 1, n) = 1$ for the smaller $M' = 420$, since $2^4 = 16 \nmid 420$ even though $2 \mid 420$; the bound must be large enough to capture the full power of each small prime dividing $p-1$, not just its presence.

1.6.4 Sieve Methods: Quadratic Sieve and GNFS (Overview)

These are the algorithms that actually matter for modern RSA key-size decisions — Pollard’s methods are pedagogically important but are not the bottleneck for well-generated 2048+ bit RSA keys.

The core idea shared by all sieve-based factoring (Fermat, QS, GNFS). Find integers x, y with $x^2 \equiv y^2 \pmod{n}$ but $x \not\equiv \pm y \pmod{n}$. Then $n \mid (x-y)(x+y)$ but $n \nmid (x-y)$ and $n \nmid (x+y)$, so $\gcd(x-y, n)$ yields a nontrivial factor. The entire game is finding such x, y efficiently — this is what distinguishes Fermat’s method (too slow to find them) from the Quadratic Sieve (clever, systematic search using smooth numbers and linear algebra over $\mathbb{F}_2$) from the General Number Field Sieve (same idea, but working in a number field for asymptotically better complexity).

Example 1.6.4 (The core relation, worked on a tiny modulus). Let $n = 35$. Notice $6^2 = 36 \equiv 1 \equiv 1^2 \pmod{35}$, so $x = 6, y = 1$ satisfies $x^2 \equiv y^2 \pmod{n}$. Check $x \not\equiv \pm y \pmod{35}$: indeed $6 \not\equiv 1$ and $6 \not\equiv -1 \equiv 34$. Then $\gcd(x-y, n) = \gcd(5, 35) = 5$ and $\gcd(x+y, n) = \gcd(7, 35) = 7$ — both nontrivial factors, recovering $35 = 5 \times 7$ instantly. QS and GNFS scale this exact idea to numbers with hundreds of digits by systematically manufacturing such (x, y) pairs from products of “smooth” relations, then solving a large linear system over $\mathbb{F}_2$ to combine them — full derivation is beyond this course, but the underlying trick is precisely this one identity.

Algorithm	Heuristic complexity	Practical relevance
Trial division	$O(\sqrt{n})$	Never, for crypto-size n
Fermat’s method	Fast only if $p \approx q$	Defense: choose p, q independently random
Pollard’s ρ	$O(n^{1/4})$	Breaks unbalanced factorizations
Pollard’s $p-1$	Fast if $p-1$ smooth	Defense: avoid smooth $p-1$ (minor concern today)
Quadratic Sieve	$L_n[1/2, 1] \approx e^{(1+o(1))\sqrt{\ln n \ln \ln n}}$	Best for n up to ~ 100 digits
GNFS	$L_n[1/3, (64/9)^{1/3}]$	Best known for large RSA moduli

Remark 1.6.5 (L -notation). $L_n[\alpha, c] = \exp(c(\ln n)^\alpha (\ln \ln n)^{1-\alpha})$ interpolates between polynomial time ($\alpha = 0$) and exponential time ($\alpha = 1$) in the bit-length of n . GNFS’s $\alpha = 1/3$ is **sub-exponential** — much better than exponential, but still infeasible at current RSA key sizes, which is exactly why 2048/3072-bit RSA remains (classically) secure.

1.6.5 Factoring Records and Key-Size Recommendations

Modulus size	Status (as of most recent public records)	Recommendation
512 bits	Factored in hours on cloud hardware	Broken, never use
768 bits	Factored (RSA-768, 2009-era GNFS effort)	Broken
829 bits (RSA-250)	Factored (2020)	Broken
1024 bits	Not publicly factored, but weak margin	Deprecated, phase out
2048 bits	No known feasible classical attack	Current baseline minimum
3072 bits	No known feasible classical attack	Recommended for long-term security
4096 bits	No known feasible classical attack	Used for high-assurance/long-lived ke

Remark 1.6.6 (Quantum caveat). *Shor’s algorithm factors n in polynomial time on a sufficiently large fault-tolerant quantum computer, breaking RSA at any classical key size. This is precisely the motivation for lattice-based (post-quantum) cryptography: lattice problems (like LWE/SVP) have no known efficient quantum algorithm, unlike factoring and discrete log.*

1.7 Summary and Bridge to Lesson 2

Summary

- RSA's correctness rests on Euler's theorem *and* CRT together — not Euler's theorem alone.
- Security rests on the RSA assumption, which is only known to be *implied by* hardness of factoring, not equivalent to it.
- Textbook (unpadded) RSA is deterministic and malleable — insecure regardless of key size.
- Practical key generation requires care: large random independent primes, avoiding smooth $p - 1$, avoiding small/related moduli across devices.
- The best classical factoring algorithm (GNFS) is sub-exponential, which is why 2048+ bit keys remain safe classically, and why quantum algorithms (Shor's) are an existential threat requiring post-quantum alternatives.

Looking Ahead — Lesson 2 — Mathematical Attacks on RSA

Next lecture assumes n itself is *not* factored, and instead breaks RSA through misuse:

- **Wiener's attack:** recovers d directly from (n, e) when d is too small, via continued fraction expansion of e/n .
- **Håstad's broadcast attack:** recovers m when the same message is sent to multiple recipients with small e , via CRT (yes — CRT again, but now as an attack tool).
- **Coppersmith's method:** recovers partial/short plaintexts or exposed key bits using lattice-based root-finding (a nice callback to lattice cryptography, if you have covered it).
- **Common modulus attack:** breaks RSA when two parties share n but use different e .

Bring: Extended Euclidean Algorithm and CRT fresh in memory — both get reused, this time as attack tools rather than construction tools.

Tutorial Exercises

1. Complete the proof of RSA correctness for the case $q \mid m$.
2. Given $p = 17$, $q = 23$, $e = 7$: compute n , $\varphi(n)$, and d . Encrypt $m = 41$ and verify decryption recovers it.
3. Explain, in your own words, why $\gcd(e, \varphi(n)) = 1$ is necessary but e being prime is not.
4. Run Pollard's ρ by hand on $n = 8051$ using $f(x) = x^2 + 1$, starting at $x_0 = 2$. Find a nontrivial factor.
5. Suppose an RSA key-generation implementation always picks q as the next prime after p (i.e., $q = \text{nextprime}(p)$). Explain concretely how you would factor such a modulus, and estimate the cost.
6. Research question: look up the RSA-250 factorization (2020) and report what algorithm was used, and how much compute time it took. Compare against a naive trial-division estimate for the same modulus size.

Lesson 2: The RSA Cryptosystem II — Mathematical Attacks

Wiener's Attack, Håstad's Broadcast Attack, Common Modulus Attacks, Franklin-Reiter, and Coppersmith's Method

Roadmap

- (1) Wiener's Attack: recovering a small private exponent d from (n, e) alone, via continued fractions
- (2) Håstad's Broadcast Attack: recovering m from low-exponent encryptions to multiple recipients, via CRT
- (3) Common Modulus Attack: breaking RSA when two parties share n but use different, coprime exponents
- (4) Cycling Attack: a conceptual (impractical) attack via the multiplicative order of c
- (5) The Franklin-Reiter Related-Message Attack: breaking two related plaintexts under the same low exponent, via polynomial GCD
- (6) Coppersmith's Method: an overview of small-root finding via lattice reduction (the LLL algorithm)

What every attack in this lecture has in common: none of them factor n . Each exploits a specific way RSA was *used* — a too-small d , a too-small e combined with related or repeated messages, exponent/modulus reuse, or the sheer algebraic structure of modular exponentiation. This is the point made in Lesson 1: breaking RSA is not the same as solving the RSA problem in general, and it is very much not the same as factoring.

2.1 Wiener's Attack: Small Private Exponent

2.1.1 Motivation

CRT-RSA decryption (Lesson 1, Section 1.2.3) is faster with smaller d . This creates a temptation: why not simply choose a small d to speed up decryption, and derive $e = d^{-1} \bmod \varphi(n)$ afterward? Wiener showed in 1990 that this is catastrophic: if d is small enough, the *entire private key* is recoverable from the public key alone, in a fraction of a second, using nothing more exotic than the Euclidean algorithm.

The key insight, in one sentence, before any machinery. The public key (n, e) is, on its face, just two large numbers with no visible structure. But if d is small, it turns out that the fraction e/n — which the attacker already knows exactly — is secretly an extremely good rational approximation of a related fraction k/d involving the private d . “Extremely good approximations of a fraction with a small denominator” are rare and easy to enumerate, so the attacker can simply search that short list rather than guessing d directly. Continued fractions are the tool that makes “search the short list of extremely good approximations” precise and efficient; we build that tool first, then use it to attack RSA.

2.1.2 Continued Fractions Refresher

What a continued fraction is, and why anyone invented it. Every rational number can be written as a whole part plus a fraction, and that fraction can itself be flipped and split the same way, recursively:

$$\frac{30}{7} = 4 + \frac{2}{7} = 4 + \frac{1}{7/2} = 4 + \frac{1}{3 + \frac{1}{2}} = 4 + \frac{1}{3 + \frac{1}{2}}.$$

Stopping this process at each stage and simplifying back into a single fraction gives a sequence of increasingly accurate rational approximations to the original number — 4, then $4 + \frac{1}{3} = \frac{13}{3}$, then the exact value $\frac{30}{7}$ itself. This is the entire idea; the definitions below just make it precise and give a fast way to compute it (the same Euclidean algorithm you already know from computing gcds and modular inverses).

Definition 2.1.1 (Continued fraction expansion). *Every rational number $\frac{x}{y}$ has a unique finite continued fraction expansion*

$$\frac{x}{y} = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\ddots + \frac{1}{a_k}}}} =: [a_0; a_1, a_2, \dots, a_k],$$

computed by the Euclidean algorithm: $a_0 = \lfloor x/y \rfloor$, then recurse on $y/(x - a_0y)$.

Example 2.1.2 (Continued fraction expansion, small warm-up). *Expand $\frac{30}{7}$. Step 1: $a_0 = \lfloor 30/7 \rfloor = 4$, remainder $30 - 4 \cdot 7 = 2$; recurse on $7/2$. Step 2: $a_1 = \lfloor 7/2 \rfloor = 3$, remainder $7 - 3 \cdot 2 = 1$; recurse on $2/1$. Step 3: $a_2 = \lfloor 2/1 \rfloor = 2$, remainder 0 — stop. So $\frac{30}{7} = [4; 3, 2]$, matching the hand-expansion above. Notice this is exactly the Euclidean algorithm run on $(30, 7)$, just recording each quotient instead of only the final gcd.*

Definition 2.1.3 (Convergents). *The i -th **convergent** $\frac{h_i}{k_i}$ of $[a_0; a_1, \dots]$ is the value of the truncated expansion $[a_0; a_1, \dots, a_i]$, computed by the recurrence*

$$h_i = a_i h_{i-1} + h_{i-2}, \quad k_i = a_i k_{i-1} + k_{i-2}, \quad (h_{-1}=1, h_{-2}=0, \quad k_{-1}=0, k_{-2}=1).$$

Example 2.1.4 (Convergents of the warm-up example). Continuing $\frac{30}{7} = [4; 3, 2]$: the convergents are

$$\frac{h_0}{k_0} = \frac{4}{1} = 4, \quad \frac{h_1}{k_1} = \frac{3 \cdot 4 + 1}{3 \cdot 1 + 0} = \frac{13}{3} \approx 4.333, \quad \frac{h_2}{k_2} = \frac{2 \cdot 13 + 4}{2 \cdot 3 + 1} = \frac{30}{7} \approx 4.286.$$

Each convergent is a better rational approximation to $\frac{30}{7} = 4.2857\dots$ than the last, and the final one is exact. This “successively better approximation” behavior, not just the recursive bookkeeping, is what Wiener’s attack actually exploits below.

Theorem 2.1.5 (Legendre’s theorem on best rational approximations). If $\left| \frac{x}{y} - \frac{h}{k} \right| < \frac{1}{2k^2}$, then $\frac{h}{k}$ is a convergent of the continued fraction expansion of $\frac{x}{y}$.

Why this matters for RSA, unpacked. Read the theorem as a converse: it does *not* say every convergent is a great approximation (that direction is easy and unsurprising); it says every *sufficiently good* approximation *must already be* one of the (few!) convergents. There are astronomically many fractions h/k with k in some range, but only $O(\log y)$ convergents — one per step of the Euclidean algorithm on (x, y) . So instead of searching an exponentially large space of candidate fractions, the theorem lets an attacker search a logarithmically short, explicitly computable list, *provided* they can first show that the fraction they are looking for happens to approximate a known quantity unusually well. That is exactly the situation Wiener’s theorem engineers next.

2.1.3 Wiener’s Theorem

The plan, stated before the algebra. The attacker knows (n, e) and wants d . The relationship defining d is $ed \equiv 1 \pmod{\varphi(n)}$, which involves the unknown $\varphi(n)$ as well as the unknown d — two unknowns, seemingly hopeless. The trick is to show that this relationship, rewritten as an inequality about *fractions*, forces e/n (fully known) to sit unusually close to k/d (fully unknown, but with small denominator d) — close enough that Legendre’s theorem applies. Once that is shown, k/d is guaranteed to appear among the convergents of e/n , which the attacker can simply compute and test one by one.

Theorem 2.1.6 (Wiener, 1990). Let $n = pq$ with $q < p < 2q$, and let $d < \frac{1}{3}n^{1/4}$. Then given only the public key (n, e) with $ed \equiv 1 \pmod{\varphi(n)}$, d can be recovered in polynomial time.

Proof, step by step. **Step 1: turn the congruence into an equation, and identify the target fraction.** Since $ed \equiv 1 \pmod{\varphi(n)}$, there is a positive integer k with

$$ed = 1 + k\varphi(n).$$

(One can check $k < d$: since $e < \varphi(n)$, we have $k = \frac{ed-1}{\varphi(n)} < \frac{ed}{\varphi(n)} < d$.) The quantity we will show is well-approximated by the known e/n is precisely k/d — a fraction built entirely from unknowns, but with denominator d , which is small by hypothesis.

Step 2: measure how far apart e/n and k/d actually are. Compute the difference directly:

$$\left| \frac{e}{n} - \frac{k}{d} \right| = \left| \frac{ed - kn}{nd} \right| = \left| \frac{(1 + k\varphi(n)) - kn}{nd} \right| = \frac{k(n - \varphi(n)) - 1}{nd}.$$

The first equality is just putting both fractions over the common denominator nd ; the second substitutes $ed = 1 + k\varphi(n)$ from Step 1; the third regroups the terms. Everything now hinges on how large $n - \varphi(n)$ is.

Step 3: bound $n - \varphi(n)$ using the balanced-primes assumption. Recall $\varphi(n) = (p-1)(q-1) = n - (p+q) + 1$, so $n - \varphi(n) = p + q - 1$. Using $q < p < 2q$ (i.e. the two

primes are within a factor of 2 of each other — the normal case for RSA key generation) gives $p + q < 3q < 3\sqrt{n} \cdot \sqrt{2}$, and a slightly more careful accounting (using $q < \sqrt{n}$ and $p < 2q$ together) gives the clean bound $n - \varphi(n) < 3\sqrt{n}$.

Step 4: combine Steps 2 and 3, and invoke the hypothesis on d . Substituting the Step 3 bound and $k < d$ into the Step 2 expression gives

$$\left| \frac{e}{n} - \frac{k}{d} \right| = \frac{k(n - \varphi(n)) - 1}{nd} < \frac{k \cdot 3\sqrt{n}}{nd} < \frac{3\sqrt{n}}{n} = \frac{3}{\sqrt{n}}.$$

A short calculation (using $d < \frac{1}{3}n^{1/4}$ to control the denominator d^2 against $\sqrt{n}$) tightens this into exactly the bound Legendre’s theorem needs: $\left| \frac{e}{n} - \frac{k}{d} \right| < \frac{1}{2d^2}$.

Step 5: invoke Legendre’s theorem. Since $\left| \frac{e}{n} - \frac{k}{d} \right| < \frac{1}{2d^2}$, Legendre’s theorem guarantees $\frac{k}{d}$ is *some* convergent of the continued fraction expansion of $\frac{e}{n}$ — a quantity the attacker already knows in full, from the public key alone, and whose convergents can be listed in $O(\log n)$ steps via the Euclidean algorithm. $\square$

What just happened, in one sentence. A relationship between two *unknowns* (e and d ’s hidden partner k) got converted, purely by algebra, into a statement that two *known* numbers (e and n) sit unusually close together as a fraction — and “unusually close” is exactly the trigger condition for Legendre’s theorem to hand the attacker a short, explicit candidate list.

Algorithm 6 Wiener’s Attack

```

1: Compute the continued fraction expansion of  $e/n$ 
2: Compute all convergents  $h_i/k_i$ 
3: for each convergent  $k/d_{\text{cand}}$  do
4:   if  $k \neq 0$  and  $(e \cdot d_{\text{cand}} - 1) \bmod k = 0$  then
5:      $\varphi_{\text{cand}} \leftarrow (e \cdot d_{\text{cand}} - 1)/k$ 
6:     Solve  $x^2 - (n - \varphi_{\text{cand}} + 1)x + n = 0$  for integer roots  $p, q$ 
7:     if integer roots exist and  $pq = n$  then
8:       return  $d = d_{\text{cand}}$  (attack succeeded)
9:     end if
10:  end if
11: end for
```

Reading the algorithm against the proof. Line 3’s convergents are exactly the candidates Step 5 guarantees contains the true k/d . Line 4 checks whether a candidate is *consistent* with being the true (k, d) : if it is, then $\varphi_{\text{cand}} = (ed_{\text{cand}} - 1)/k$ recomputes what $\varphi(n)$ would have to be. Line 6 then uses φ_{cand} and n together to solve for p, q directly — since $p + q = n - \varphi_{\text{cand}} + 1$ and $pq = n$ are two equations in two unknowns, giving a quadratic. If that quadratic has integer roots multiplying to n , the candidate was correct; if not, move to the next convergent. Because only $O(\log n)$ convergents exist, this whole search is fast even though it looks like a brute-force loop.

Example 2.1.7 (Wiener’s attack, fully worked). Let $p = 1223$, $q = 1987$, so $n = 2,430,101$ and $\varphi(n) = 2,426,892$. Suppose the key-generator (unwisely) chose a small private exponent $d = 25$ for decryption speed. Then $e = d^{-1} \bmod \varphi(n) = 2,135,665$ — this is the only thing published.

Check the bound: $\frac{1}{3}n^{1/4} \approx 13.16$. Wait — $d = 25 > 13.16$, so strictly this instance sits outside Wiener’s original (conservative) bound; we use it anyway because the attack succeeds in practice on this instance regardless (the bound is a sufficient, not tight necessary, condition — later refinements by Boneh–Durfee push the provable bound up to $d < n^{0.292}$, which comfortably covers this example). This is worth flagging explicitly to students: textbook bounds are usually conservative sufficient conditions, not sharp thresholds.

Run the attack: the continued fraction expansion of $e/n = 2135665/2430101$ is

$$[0; 1, 7, 3, 1, 17, 1, 1, 2, 1, 2, 211].$$

Its convergents h_i/k_i include (among others) $\frac{22}{25}$. Testing $k = 22, d_{\text{cand}} = 25$: $(e \cdot 25 - 1) \bmod 22 = 0$, giving $\varphi_{\text{cand}} = 2,426,892$ — exactly right. Solving the resulting quadratic for p, q recovers $p = 1223, q = 1987$ **exactly**. The attacker recovered the entire private key — not just d , but the factorization of n itself — using only the public key (n, e) and a handful of Euclidean-algorithm steps.

Practical consequence. Never choose d for efficiency and derive e afterward. Standard practice is the reverse: fix a large, well-understood e (e.g., 65537) and compute $d = e^{-1} \bmod \varphi(n)$, which is essentially always large. If CRT-RSA speed is the actual goal, achieve it via the CRT decomposition of Lesson 1 (Section 1.2.3) instead — that optimization does not shrink d itself, only the size of the exponents used *internally* during decryption.

2.2 Håstad's Broadcast Attack

2.2.1 Setup

Suppose a sender uses a **small public exponent** (e.g., $e = 3$) and sends the *same* message m , encrypted under e , to e or more different recipients with pairwise coprime moduli $n_1, \dots, n_e$ (their own independent RSA keys):

$$c_i = m^e \bmod n_i, \quad i = 1, \dots, e.$$

Why this should feel dangerous, before the proof. Recall the “no modular wraparound” example from Lesson 1: if $m^e < n$, then $c = m^e$ is simply the integer m^e with no reduction ever taking place, and an ordinary integer root recovers m instantly. A single well-generated RSA key avoids this by making n enormous compared to any realistic m^e . Håstad's insight is that broadcasting the *same* message to *several* recipients recreates exactly this vulnerability one level up: even though each individual $c_i = m^e \bmod n_i$ wraps around plenty, the Chinese Remainder Theorem lets an eavesdropper combine all e ciphertexts into a single value modulo the much larger product $N = n_1 \cdots n_e$ — and against that enormous combined modulus, m^e itself might not wrap around at all. Three small worlds, stitched together by CRT, behave like one giant world in which the “no wraparound” flaw resurfaces.

Theorem 2.2.1 (Håstad's broadcast attack, $e = 3$ identical-message case). *If $m < \min(n_1, n_2, n_3)$, an eavesdropper who observes c_1, c_2, c_3 (and the public moduli) recovers m in polynomial time, without knowing any private key.*

Proof, step by step. **Step 1: combine the three congruences with CRT.** The eavesdropper knows c_1, c_2, c_3 and the public n_1, n_2, n_3 (pairwise coprime), and wants a single x with $x \equiv c_i \pmod{n_i}$ for $i = 1, 2, 3$. By CRT (Lesson 1, Theorem 1.1.14, extended to three coprime moduli the same way it works for two), this system has a *unique* solution C modulo $N = n_1 n_2 n_3$, and — crucially — CRT is constructive: C is computable directly from the public c_i, n_i values using the reconstruction formula from Lesson 1, no private key involved anywhere.

Step 2: notice m^3 itself satisfies the same three congruences. Since $c_i \equiv m^3 \pmod{n_i}$ for each i by definition of encryption, the value m^3 is *also* a solution to the system in Step 1. By the uniqueness half of CRT, there is only one solution modulo N — so $C \equiv m^3 \pmod{N}$.

Step 3: upgrade “congruent mod N ” to “equal as integers”. This is the step where the size assumption $m < \min(n_1, n_2, n_3)$ earns its keep: it gives $m^3 < n_1 n_2 n_3 = N$. Since C (as CRT constructs it) is itself a specific representative in $\{0, \dots, N-1\}$, and m^3 is a nonnegative integer strictly less than N that is congruent to C modulo N , the two must be *exactly* the same integer, not merely congruent — there is no room for them to differ by a multiple of N . So $C = m^3$ holds as an honest equation over $\mathbb{Z}$, not just modulo N .

Step 4: take an ordinary integer cube root. Since C is now known to be exactly m^3 (an integer, not a residue), an ordinary integer cube-root computation — no modular arithmetic, no private key — recovers m . $\square$

The one-sentence summary. Three separate “small worlds” ($\bmod n_1$, $\bmod n_2$, $\bmod n_3$), each individually opaque, get stitched by CRT into one “big world” ($\bmod N = n_1 n_2 n_3$) in which the encryption exponent's usual protection — reduction mod a modulus much larger than the message — silently stops applying.

Example 2.2.2 (Håstad's attack, $e = 3$, fully worked). *Let three (toy) recipients have moduli $n_1 = 143$, $n_2 = 323$, $n_3 = 667$ (pairwise coprime by construction: $143 = 11 \cdot 13$, $323 = 17 \cdot 19$, $667 = 23 \cdot 29$ — no shared prime factors). A sender broadcasts $m = 17$ to all three under $e = 3$:*

$$c_1 = 17^3 \bmod 143 = 51, \quad c_2 = 17^3 \bmod 323 = 68, \quad c_3 = 17^3 \bmod 667 = 244.$$

An eavesdropper computes $N = n_1 n_2 n_3 = 30,808,063$ and solves the CRT system to get $C = 4913$. Since $m^3 = 17^3 = 4913 < N$, this C is the true integer value of m^3 (no wraparound occurred). Taking the integer cube root: $\sqrt[3]{4913} = 17 = m$. **No private key was ever touched.**

Remark 2.2.3 (Generalizing beyond identical messages). *The identical-message case above is the simplest illustration. If instead each recipient gets a linearly related message (e.g., $m_i = a_i m + b_i$ for known public a_i, b_i — a common real-world pattern, since padding schemes or per-recipient headers often relate messages this way), the same underlying idea extends via a more general CRT-plus-resultant construction. This generalization is due to Håstad, and its unknown-relation extension (where even the relationship between messages is not fully known, only that it is small) is precisely Coppersmith’s short-pad attack, previewed at the end of this lecture and building on the lattice technique in Section 2.6.*

Practical consequence. **Never** use a small public exponent to send the same (or simply related) message to multiple recipients without randomized padding. OAEP (Lesson 3) defeats this attack by making every ciphertext of the same message under the same key look independently random, so no algebraic relationship exists across ciphertexts for CRT to exploit.

2.3 Common Modulus Attack

2.3.1 Setup

A tempting (and wrong) efficiency idea: have a central authority generate a single modulus n , and give each user their own (e_i, d_i) pair on the *same* n , to save on key-generation cost. Suppose a message m is (perhaps accidentally, perhaps because the sender does not know which key to use) encrypted under two different users' exponents e_1, e_2 on the shared n , with $\gcd(e_1, e_2) = 1$:

$$c_1 = m^{e_1} \bmod n, \quad c_2 = m^{e_2} \bmod n.$$

Why this should feel dangerous, before the proof. Think about what $\gcd(e_1, e_2) = 1$ actually buys you: by the Extended Euclidean Algorithm (the same tool from Lesson 1 that computes $d = e^{-1} \bmod \varphi(n)$), there exist integers a, b with $ae_1 + be_2 = 1$. That is a way of writing the number 1 itself as a combination of e_1 and e_2 . Since c_1 and c_2 are both just m raised to a power, and exponents combine by *multiplying* when you raise a power to a power, “combining e_1 and e_2 to make 1” translates directly into “combining c_1 and c_2 to make $m^1 = m$ ” — no private key needed at any point, because everything used here (e_1, e_2, c_1, c_2, n) is public.

Theorem 2.3.1 (Common modulus attack). *If $\gcd(e_1, e_2) = 1$, an eavesdropper who knows only (n, e_1, e_2, c_1, c_2) — no private key at all — recovers m .*

Proof, step by step. **Step 1: find Bézout coefficients for e_1, e_2 .** Since $\gcd(e_1, e_2) = 1$, the Extended Euclidean Algorithm produces integers a, b (exactly one of which will typically be negative) with

$$ae_1 + be_2 = 1.$$

Step 2: raise each ciphertext to the matching coefficient, and multiply. Compute $c_1^a c_2^b \bmod n$ and expand using the definitions $c_1 = m^{e_1}$, $c_2 = m^{e_2}$:

$$c_1^a c_2^b \equiv (m^{e_1})^a (m^{e_2})^b \equiv m^{ae_1} \cdot m^{be_2} \equiv m^{ae_1 + be_2} \pmod{n}.$$

The first equality substitutes the definitions of c_1, c_2 ; the second and third just apply the exponent laws $x^{mn} = (x^m)^n$ and $x^m x^n = x^{m+n}$ — nothing here uses any secret.

Step 3: substitute Step 1's identity into the exponent. Since $ae_1 + be_2 = 1$ exactly (an integer identity, not just a congruence), the exponent in Step 2 collapses:

$$m^{ae_1 + be_2} = m^1 = m.$$

Chaining Steps 2 and 3 gives $c_1^a c_2^b \equiv m \pmod{n}$ — the plaintext, recovered from two public ciphertexts and two public exponents alone.

Step 4: handle a negative exponent. If (say) $b < 0$, then c_2^b is not directly computable by repeated squaring in the usual sense; instead compute $c_2^{-1} \bmod n$ first via the Extended Euclidean Algorithm (this requires $\gcd(c_2, n) = 1$, true with overwhelming probability for a random message — the exceptional case, $\gcd(c_2, n) \neq 1$, would itself immediately leak a factor of n via a single gcd computation, an even worse outcome for the key owner), and then raise that inverse to the power $|b|$ instead. $\square$

The one-sentence summary. Two exponentiations of the *same* secret message, under two exponents that are coprime, can always be algebraically recombined into the message itself, because coprimality is exactly the condition under which 1 can be written as an integer combination of the two exponents.

Example 2.3.2 (Common modulus attack, fully worked). Let $n = 3233$ (our familiar $p = 61, q = 53$ toy modulus). Two users share n with $e_1 = 17, e_2 = 7$ (coprime). A message $m = 42$ is encrypted under both: $c_1 = 42^{17} \bmod 3233 = 2557, c_2 = 42^7 \bmod 3233 = 240$.

Extended Euclid on $(17, 7)$ gives $a = -2, b = 5$ (check: $-2 \cdot 17 + 5 \cdot 7 = -34 + 35 = 1$). Since $a < 0$, compute $c_1^{-1} \bmod n = 3123$ (via modular inverse), then

$$m = (c_1^{-1})^2 \cdot c_2^5 \bmod n = 42.$$

Recovered exactly — **using no private key whatsoever**, just the two public exponents and two ciphertexts.

Practical consequence. Never share a modulus n across users or purposes, even with different exponents. This design pattern occasionally resurfaces in embedded/legacy systems trying to save key-generation cost, but it is fundamentally broken the moment any two exponents used on the shared modulus are coprime — which is the common case, since exponents are typically chosen small and prime.

2.4 Cycling Attack (Conceptual)

2.4.1 Setup

This attack requires no assumption about d , e , or shared moduli — only encryption itself, and is presented mainly for conceptual completeness (it is essentially never practical at real key sizes, but it is instructive about the group-theoretic structure of RSA).

Why this should feel familiar, before the proof. Recall from Lesson 1 that repeatedly multiplying a group element a by itself eventually returns to 1 — that was the whole content of Euler’s theorem, proved there by showing multiplication-by- a just permutes the group. *Repeated encryption* is exactly repeated exponentiation by the same fixed exponent e , so the same cyclic idea applies one level up: instead of asking “how many multiplications by a get back to 1?” we ask “how many *encryptions* (exponentiations by e) get back to the original ciphertext?” Because encryption is invertible (that is the entire point of RSA), this cycle must eventually close, and — remarkably — the plaintext sits in plain view one step before it closes.

Theorem 2.4.1 (Cycling attack). *Given only (n, e, c) where $c = m^e \bmod n$, repeatedly compute $c_1 = c^e \bmod n$, $c_2 = c_1^e \bmod n$, and so on, until some $c_k = c$. Then $m = c_{k-1}$.*

Proof, step by step. **Step 1: express every term of the sequence as a single power of m .** Encrypting repeatedly under the public exponent is the same as encrypting-then-encrypting, so writing $c_0 = c$, an easy induction gives $c_i \equiv m^{e^{i+1}} \pmod{n}$ for every $i \geq 0$ (each step multiplies the current exponent of m by another factor of e).

Step 2: define k as the point where the sequence first returns to its start. Let k be the smallest positive integer with $c_k \equiv c_0 \pmod{n}$. By Step 1, this says $m^{e^{k+1}} \equiv m^e \pmod{n}$.

Step 3: cancel one factor of e from the exponent, using invertibility. Assuming $\gcd(m, n) = 1$, the congruence $m^{e^{k+1}} \equiv m^e \pmod{n}$ is really a statement about the *order* of m in the group $(\mathbb{Z}/n\mathbb{Z})^*$ (Lesson 1’s group again): it says $e^{k+1} \equiv e \pmod{\text{ord}_n(m)}$. Recall e was chosen coprime to $\varphi(n)$ during key generation, and $\text{ord}_n(m)$ always divides $\varphi(n)$ (an element’s order divides the group’s size — this is the same Lagrange-style fact behind Euler’s theorem), so e is automatically coprime to $\text{ord}_n(m)$ too, and therefore invertible modulo $\text{ord}_n(m)$. Multiplying both sides of $e^{k+1} \equiv e$ by $e^{-1} \bmod \text{ord}_n(m)$ cancels one factor of e cleanly:

$$e^k \equiv 1 \pmod{\text{ord}_n(m)}.$$

Step 4: read off the plaintext. Substituting this back, the term one step before the cycle closes is

$$c_{k-1} \equiv m^{e^k} \equiv m^1 \equiv m \pmod{n},$$

using Step 3 exactly as Lesson 1 used Euler’s theorem: raising to an exponent that is $\equiv 1$ modulo the element’s order leaves the element unchanged. $\square$

The one-sentence summary. Repeated encryption is iteration inside a finite cyclic structure — it *must* eventually loop back to the starting ciphertext, and by the same “one full lap plus one step” logic from Lesson 1’s Euler’s-theorem walkthrough, the plaintext is sitting exactly one step before the loop closes.

Example 2.4.2 (Cycling attack, fully worked). *Let $p = 47, q = 59$, so $n = 2773$, $\varphi(n) = 2668$, and let $e = 17$. A message $m = 123$ is encrypted: $c = 123^{17} \bmod 2773 = 1924$.*

*Repeatedly raising to the e -th power: $c_1 = c^{17} \bmod n, c_2 = c_1^{17} \bmod n, \dots$ — after **44** iterations, the sequence returns to $c_{44} = 1924 = c$. The value one step before, $c_{43} = 123 = m$, is the recovered plaintext.*

Why this stays purely conceptual. The cycle length here is the multiplicative order of e in $(\mathbb{Z}/\lambda(n)\mathbb{Z})^*$, which for well-chosen keys (large random primes, standard e) is astronomically large — comparable to $\varphi(n)$ itself in the worst case for the attacker. At real key sizes this attack is utterly infeasible; it survives in courses because it cleanly demonstrates that RSA encryption is literally iteration inside a finite cyclic structure, and it foreshadows why key-generation standards historically recommended checking that $\lambda(n)$ has large prime factors (to keep every element's order large) — the same underlying concern as smooth $p - 1$ in Pollard's method (Lesson 1).

2.5 The Franklin–Reiter Related-Message Attack

2.5.1 Setup

Suppose the *same recipient* (same n, e) receives two ciphertexts of two messages known to satisfy a **publicly known linear relationship** $m_2 = m_1 + t$ for some known constant t (e.g., two near-identical form-letter messages differing only in a numeric field), both under a small exponent, e.g. $e = 3$:

$$c_1 = m_1^3 \bmod n, \quad c_2 = (m_1 + t)^3 \bmod n.$$

Why this should feel familiar, before the proof. Every earlier attack in this lecture combined two *numbers* via the Euclidean algorithm (Wiener: numerator/denominator pairs; common modulus: two exponents) to isolate an unknown. Franklin–Reiter does the same trick one level of abstraction up: instead of combining two *integers*, it combines two *polynomials* — and the “Euclidean algorithm” for polynomials (repeated division with remainder, exactly as with integers, just tracking coefficients instead of digits) plays exactly the same role. The key structural fact is that both polynomials constructed below happen to share m_1 as a root purely by how they were built from public information — so their greatest common divisor must contain the factor $(x - m_1)$, and generically nothing else survives.

Theorem 2.5.1 (Franklin–Reiter, 1996). *Given $(n, e=3, t, c_1, c_2)$, m_1 can be recovered in polynomial time, without factoring n .*

Proof, step by step. **Step 1: build two polynomials that are both public knowledge to construct, and both have m_1 as a root.** Define, over $\mathbb{Z}_n[x]$ (polynomials with coefficients reduced mod n — everything needed to write these down is public):

$$f_1(x) = x^3 - c_1, \quad f_2(x) = (x + t)^3 - c_2.$$

Substituting $x = m_1$ into f_1 gives $m_1^3 - c_1 = 0$ by the definition of c_1 ; substituting $x = m_1$ into f_2 gives $(m_1 + t)^3 - c_2 = 0$ by the definition of c_2 . So $x = m_1$ is a genuine root of *both* polynomials, even though the attacker does not know m_1 — only that it exists as a common root.

Step 2: a shared root means $(x - m_1)$ divides the gcd. Basic polynomial algebra: if $x = m_1$ is a root of a polynomial f , then $(x - m_1)$ divides $f(x)$ exactly (polynomial division by a linear factor with zero remainder — the polynomial analogue of “if a divides b with remainder 0”). Since $(x - m_1)$ divides both f_1 and f_2 , it divides any combination the Euclidean algorithm produces from them, and in particular it divides $\gcd(f_1, f_2)$.

Step 3: argue the gcd is *exactly* $(x - m_1)$, generically. f_1 has degree 3 and f_2 has degree 3; their gcd could, in principle, be larger than the single factor $(x - m_1)$ if the two cubics happened to share some *other* root too. But for two cubics built from unrelated random-looking ciphertexts, sharing a second common root “by accident” happens with negligible probability — so in essentially every real instance, $(x - m_1)$ *is* the entire gcd, up to an irrelevant constant multiple. A linear polynomial’s root is read off immediately.

Step 4: compute the gcd mechanically, via the polynomial Euclidean algorithm. Exactly as with integers — repeatedly divide the higher-degree polynomial by the lower-degree one and keep the remainder, until the remainder’s degree can drop no further — except now every arithmetic operation on coefficients (add, subtract, multiply, and crucially *divide*, which needs a modular inverse) is carried out modulo n . The process terminates at a linear polynomial $\alpha x + \beta \equiv 0 \pmod{n}$, whose unique root $x \equiv -\beta\alpha^{-1} \pmod{n}$ is m_1 . $\square$

The one-sentence summary. Two polynomials, built entirely from public data, are engineered by construction to share exactly one common root — the secret message — and the

same Euclidean-algorithm machinery that finds integer gcds finds polynomial gcds just as mechanically, extracting that shared root directly.

Example 2.5.2 (Franklin–Reiter, fully worked). *Let $n = 3233$, $e = 3$, and suppose $m_1 = 20$, with a second message $m_2 = m_1 + t = 25$ for known offset $t = 5$. Ciphertexts: $c_1 = 20^3 \bmod 3233 = 1534$, $c_2 = 25^3 \bmod 3233 = 2693$.*

*Build $f_1(x) = x^3 - 1534$ and $f_2(x) = (x + 5)^3 - 2693 = x^3 + 15x^2 + 75x + (125 - 2693)$ over $\mathbb{Z}_{3233}[x]$. Running the polynomial Euclidean algorithm (dividing f_1 by f_2 , then continuing with remainders, all coefficient arithmetic done mod 3233) reduces the pair to a single **linear** polynomial: $525x + 2432 \equiv 0 \pmod{3233}$. Solving: $x \equiv -2432 \cdot 525^{-1} \equiv 20 \pmod{3233}$ — exactly m_1 . **Neither n was factored nor any private key touched** — only public ciphertexts, the known offset t , and polynomial arithmetic.*

Practical consequence. **Never** encrypt structurally related plaintexts (form letters, sequential IDs, incrementing counters) under a small exponent without randomized padding. This is one of the strongest practical arguments for OAEP: it destroys any algebraic relationship between the padded plaintexts, even when the underlying messages are related, which is exactly what this attack (and its harder unknown-relation cousin below) depends on.

2.6 Coppersmith’s Method: An Overview

2.6.1 The General Problem

What problem is being solved, and why it is hard in general. Every attack so far in this lecture found an unknown by exploiting some special structure of the specific scenario (small d , shared modulus, related messages). Coppersmith’s method attacks a much more general question directly: *given a polynomial equation that holds modulo n , when can we find a root, even without knowing how to factor n ?* Solving polynomial equations modulo a composite number of unknown factorization is, in general, believed to be hard — it is essentially as hard as factoring n itself for a typical polynomial. Coppersmith’s remarkable result is that this hardness has an exception precisely when the root you are looking for is *unusually small* relative to n and the polynomial’s degree.

Coppersmith’s small-root theorem (informal statement). Given a polynomial $f(x)$ of degree δ modulo n (with n of unknown or partially unknown factorization), if f has a root x_0 with $|x_0| < n^{1/\delta}$, then x_0 can be found in time polynomial in $\log n$ and δ .

Why smallness is exactly the right lever. Think of it this way: an arbitrary root modulo n carries roughly $\log_2 n$ bits of information — as much as n itself — so pinning it down should be exactly as hard as factoring. But a root promised to satisfy $|x_0| < n^{1/\delta}$ carries only about $\frac{1}{\delta} \log_2 n$ bits — a small fraction of a full-size unknown. Coppersmith’s theorem says that whenever the *unknown part* of the problem is this small, there is enough redundancy in the surrounding structure (made precise by the lattice construction below) to pin it down efficiently, without ever needing to factor n along the way.

This single theorem — due to Coppersmith (1996), with a simplified proof by Howgrave-Graham — underlies an entire family of RSA attacks that all reduce to “find a small root of a polynomial modulo n ”:

- **Håstad’s short-pad attack** (the unknown-relation generalization of Håstad’s broadcast attack above): recovers m when two related messages are sent under low e even when the *relationship itself* is not fully known, only that it is small.
- **Partial key exposure**: given enough of the high-order (or low-order) bits of p , recover the rest of p and factor n — directly relevant to side-channel and cold-boot attacks where an attacker recovers a fraction of the key bits.
- **Boneh–Durfee**: an improvement on Wiener’s bound, extending the recoverable range of small d from $n^{0.25}$ up to $n^{0.292}$, using a two-variable version of Coppersmith’s technique.
- **Stereotyped-message attacks**: recovering m when only a small unknown portion of a structured plaintext (e.g., a fixed-format message with one unknown short field) is unknown.

The common shape of all four. Look closely and every bullet above is the same sentence with different nouns substituted in: “most of some quantity is known; a small remaining unknown piece satisfies a polynomial relationship mod n ; recover the small piece.” Recognizing a new attack scenario as an instance of this one shape is the single most useful skill Coppersmith’s method teaches — the hard mathematical work (below) only has to be understood once.

2.6.2 The Reduction to Lattices

The high-level plan, before the machinery. A direct search for x_0 modulo n is hard because n ’s factorization is unknown — ordinary polynomial root-finding techniques need to work mod

a prime or over the reals, not mod a mystery composite. Coppersmith’s trick sidesteps this entirely: instead of solving the polynomial equation modulo n directly, build *several* polynomials that are all guaranteed to share the root x_0 modulo n , and search for a clever *integer* combination of them that is small enough to force the same equation to hold as an honest equation over the integers $\mathbb{Z}$ — a domain where root-finding is easy and needs no knowledge of n ’s factors at all. “Searching for a clever small combination” is precisely the Shortest Vector Problem in disguise, which is why lattice reduction enters the picture.

From polynomial roots to short vectors. The key trick: build several polynomials $g_1(x), g_2(x), \dots, g_k(x)$, all sharing the same root $x_0 \bmod n$ (typically shifts and powers of the original f , multiplied by powers of n to control the modulus), and treat their *coefficient vectors* as a basis for a lattice $L \subset \mathbb{Z}^k$. A theorem of Howgrave-Graham says: if $h(x) = \sum h_i x^i$ is an integer linear combination of the g_i (hence still has x_0 as a root mod n) with **small enough Euclidean norm** $\|h\| < n/\sqrt{k}$, then $h(x_0) = 0$ holds as an equation over $\mathbb{Z}$, *not just modulo n* . Since h is now known to vanish at x_0 as an ordinary integer polynomial, standard root-finding recovers x_0 directly.

Why the norm bound is exactly the right threshold — the intuition behind Howgrave-Graham’s lemma. If $h(x_0) \equiv 0 \pmod{n}$, then $h(x_0)$ is *some* multiple of n — possibly 0, possibly n , possibly $-2n$, and so on. If we can additionally guarantee $h(x_0)$ is smaller in absolute value than n itself, the only multiple of n small enough to fit is 0 — forcing $h(x_0) = 0$ exactly. Bounding $|h(x_0)|$ in terms of h ’s coefficient sizes (via the Euclidean norm $\|h\|$) and the known bound on x_0 is exactly what turns “small coefficients” into “small value at x_0 ”, which is why finding a short vector in the lattice L is the right computational target.

So the whole problem reduces to: **find a short vector in the lattice L** spanned by the g_i ’s coefficient vectors. This is precisely Shortest Vector Problem territory — and in practice, the LLL algorithm (an approximate-SVP solver) is exactly the tool used, since Coppersmith’s bound is calibrated to what LLL can reliably deliver in polynomial time. If you have covered lattice-based cryptography, this is the same LLL you met there, now redeployed as an attack tool rather than a hardness-assumption underpinning.

Example 2.6.1 (Toy illustration of the core mechanism — small-scale, not full LLL). *Suppose we know p shares its top bits with a known guess $\tilde{p}$: specifically $p = \tilde{p} + x_0$ for some small unknown x_0 , and we are told $|x_0| < n^{1/2}$ (deliberately generous, purely for illustration). Consider $f(x) = \tilde{p} + x$, whose root mod p is x_0 by construction. In the real attack, one builds a small lattice from shifted copies of f , powers of n or p , and known bounds on x_0 , then runs LLL to extract a short combination $h(x)$ satisfying Howgrave-Graham’s norm bound; solving $h(x_0) = 0$ over $\mathbb{Z}$ (an ordinary, easy root-finding step once h is small) recovers x_0 , hence p , hence the full factorization of n .*

*At the tiny numeric scale used throughout this lecture (3–4 digit moduli), the lattices involved collapse to dimension 1 or 2 and the bound on x_0 needed for a compelling illustration is not usefully separable from simple brute-force search — so a genuinely small worked numeric example does not illustrate the mechanism honestly. This is a case where **the technique’s value only appears at realistic scale** (hundreds of bits), which is exactly why Coppersmith’s method is taught as a reduction to a problem you already have a tool for (LLL), rather than by hand-computation as in the earlier attacks in this lecture.*

Remark 2.6.2 (Why this belongs at the end of this lecture, not the start). *Every attack earlier in this lecture (Wiener, Håstad, common modulus, Franklin–Reiter) can be fully hand-verified with small numbers, which is why they came first. Coppersmith’s method is the most powerful of the family but the least hand-computable — its correctness rests on LLL actually finding a short-enough vector, which is a statement about lattice reduction quality, not elementary number theory. This is intentionally the conceptual capstone of RSA’s mathematical attacks, and it is*

the point where number theory hands off to lattice reduction (the LLL algorithm) as the working tool.

2.7 Summary and Bridge to Lesson 3

Summary

- **Wiener's attack:** a small d is recoverable directly from (n, e) via continued fractions — never choose d for efficiency.
- **Håstad's broadcast attack:** low exponent + repeated/related messages to multiple recipients + no padding = full break via CRT.
- **Common modulus attack:** sharing n across users with coprime exponents is a full break via the Extended Euclidean Algorithm.
- **Cycling attack:** a conceptual reminder that RSA encryption is iteration in a finite group — impractical, but foundational.
- **Franklin-Reiter:** known-relation messages under low exponent break via polynomial GCD.
- **Coppersmith's method:** the general small-root machinery underlying stronger versions of several of the above, reducing to LLL lattice reduction.

The unifying lesson: every attack here targets a specific *misuse* of RSA — small exponents without padding, small private exponents, repeated moduli, related plaintexts. None of them factor n , and all of them are defeated by proper randomized padding (OAEP) and standard key-generation discipline (large random d , non-shared n , large e such as 65537).

Looking Ahead — Lesson 3 — Implementation and Protocol Attacks on RSA

Next lecture assumes RSA is used *correctly* at the mathematical level (proper key generation, no shared moduli, no related-message broadcasting) and instead breaks it through its physical or protocol-level implementation:

- **Bleichenbacher's padding oracle attack** on PKCS#1 v1.5 — recovering plaintext by observing only whether a server accepts or rejects a padding format.
- **Timing attacks** exploiting the data-dependent branching in square-and-multiply (flagged back in Lesson 1).
- **Fault attacks** on CRT-RSA (the Bellcore attack) — a single computational glitch during the m_p or m_q branch (Lesson 1, Section 1.2.3) that factors n from one faulty signature.
- **ROCA** and other real-world key-generation vulnerabilities.
- OAEP's security argument, and why it defeats every low-exponent attack in today's lecture.

Tutorial Exercises

1. Run Wiener's attack by hand on $n = 100822548703$ with $e = 74531608157$ — hint: the continued fraction expansion has fewer than 10 terms. (Verify your recovered d by checking $ed \equiv 1$ modulo your recovered $\varphi(n)$.)
2. In the Håstad broadcast example, what is the minimum number of recipients needed if $e = 5$ instead of 3? Explain why in terms of the CRT bound $m^e < N$.

3. Extend the common modulus attack: what happens if $\gcd(e_1, e_2) = g > 1$? Is the attack still possible, and if so what can be recovered?
4. In the cycling attack, express the cycle length k precisely in terms of the multiplicative order of e modulo $\lambda(n)$ (the Carmichael function), and explain why choosing p, q such that $\lambda(n)$ has a large prime factor defends against short cycles.
5. Redo the Franklin–Reiter worked example with $e = 3$, $t = 7$, $m_1 = 30$ on the same $n = 3233$. Verify your recovered m_1 .
6. Research question: read the original Boneh–Durfee paper’s abstract and result statement. What is the improved bound on d compared to Wiener’s $n^{0.25}$, and what extra mathematical tool (beyond one-dimensional continued fractions) does it require?

Lesson 3: The RSA Cryptosystem III — Implementation and Protocol Attacks

Padding Oracles, Timing and Fault Attacks, ROCA, OAEP, and the Guillou–Quisquater ZKP Bridge

Roadmap

- (1) Padding oracles: PKCS#1 v1.5 and Bleichenbacher’s “million-message” attack
- (2) Timing attacks: leaking secret exponent bits through execution time, and blinding as the fix
- (3) Fault attacks: the Bellcore attack on CRT-RSA signatures
- (4) ROCA: a real-world key-generation flaw that turned Coppersmith’s method (Lesson 2) into a practical factoring attack
- (5) Side-channel countermeasures: a consolidated checklist
- (6) OAEP: a security sketch for the padding scheme that defeats almost everything in Lessons 2–3
- (7) Guillou–Quisquater: an RSA-based zero-knowledge proof of identity, and why it needs only one exponentiation-sized challenge space instead of many rounds

What every attack in this lecture has in common: n is properly generated, d is large, no modulus is shared, no message is broadcast — every mathematical precondition from Lessons 1–2 is satisfied. These attacks instead exploit *how RSA is computed* (timing, faults) or *how a protocol reacts to malformed input* (padding oracles), or a subtle flaw in *how the keys themselves were generated* (ROCA). This is the layer of RSA security that pure number theory cannot see.

Why this lecture feels different from the last two. In Lessons 1–2, every attack lived entirely on paper: given some numbers, do some algebra, get the secret. Here, the “numbers” are produced by a real physical computer running real code, and that computer takes measurable time, draws measurable power, and can be made to make mistakes (deliberately, by an attacker). The mathematics you already know (CRT, Euler’s theorem, Coppersmith’s method) does not change — it gets *reused*, but now the input to that mathematics is something the attacker extracted from the outside world rather than something sitting in a public key file. Keep asking, for each attack below: *what physical signal is being turned into a piece of secret information, and by which piece of Lesson 1–2 mathematics is that piece then exploited?*

3.1 Padding Oracles: Bleichenbacher’s Attack on PKCS#1 v1.5

3.1.1 PKCS#1 v1.5 Encryption Padding

Recall from Lesson 1 that textbook RSA is deterministic and malleable, and that real systems pad the message before encrypting. The older padding standard, **PKCS#1 v1.5**, formats a k -byte block (where k is the byte-length of n) as

$$EB = 0x00 \parallel 0x02 \parallel PS \parallel 0x00 \parallel D,$$

where D is the message, PS is a string of *non-zero* random padding bytes at least 8 bytes long (padding out the block to exactly k bytes), and the leading $0x00 \ 0x02$ is a fixed, publicly-known format marker. The recipient decrypts c , obtains $EB = c^d \bmod n$ (represented as a k -byte string), and checks the format: does it start with $0x00 \ 0x02$, is there a $0x00$ byte terminating the padding within a valid range, etc. If the format is invalid, many real implementations (TLS servers being the classic case) return a distinguishable error — *this is the oracle*.

Definition 3.1.1 (Padding oracle). A **padding oracle** for PKCS#1 v1.5 is any mechanism (an error message, a timing difference, a connection reset — the specific channel does not matter) that lets an attacker submit a chosen ciphertext c and learn a single bit: does $c^d \bmod n$ decrypt to a validly-PKCS#1-padded block, i.e. does it start with $0x00 \ 0x02$?

Example 3.1.2 (A tiny PKCS#1 v1.5 block, made concrete). Take a toy $k = 8$ -byte block (a real RSA-1024 block would use $k = 128$; 8 bytes is enough to see the structure). Encoding the 2-byte message $D = 0x1234$ with 4 bytes of random non-zero padding $PS = 0xA1 \ 0x7F \ 0x3C \ 0x99$ gives

$$EB = \underbrace{0x00}_{1 \text{ byte}} \underbrace{0x02}_{1 \text{ byte}} \underbrace{0xA1 \ 0x7F \ 0x3C \ 0x99}_{PS, 4 \text{ bytes}} \underbrace{0x00}_{1 \text{ byte}} \underbrace{0x1234}_{\text{message, 2 bytes}},$$

exactly 8 bytes. A recipient decrypting some ciphertext c' and obtaining, say, $0x00 \ 0x02 \ 0x00 \dots$ (a zero byte immediately after the $0x02$ marker, where PS must be non-zero) rejects it as invalid — this is precisely the format check the padding oracle exposes as a single accept/reject bit, with nothing about the message D itself revealed directly by the check.

Why one bit is catastrophic — the mechanism, unpacked. A one-bit oracle sounds harmless — surely one bit of information about a giant secret is nothing? Bleichenbacher (CRYPTO 1998) showed the opposite, and the mechanism has two ingredients worth separating clearly.

Ingredient 1: RSA lets you scale the hidden message by a chosen factor, without knowing it. Recall RSA’s malleability from Lesson 1: given $c = m^e \bmod n$, anyone can compute $c \cdot s^e \bmod n$ for a chosen public s , and this is a valid encryption of $m \cdot s \bmod n$ — because $c \cdot s^e \equiv m^e s^e \equiv (ms)^e \pmod{n}$, pure exponent-law algebra, no secret required. So an attacker who only ever gets a single ciphertext c can still generate an unlimited family of *related* ciphertexts $c \cdot s^e \bmod n$, one for every s they choose, each corresponding to the unknown m scaled by that chosen s .

Ingredient 2: the oracle turns “is $ms \bmod n$ in a certain narrow range” into a one-bit test. The padding format requires the decrypted block to start with $0x00 \ 0x02$, which — read as a number — means the decrypted value must fall in a specific narrow range $[2B, 3B)$ out of all of $\{0, \dots, n - 1\}$, where $B = 2^{8(k-2)}$. So submitting $c \cdot s^e \bmod n$ to the oracle and getting a YES answer tells the attacker, for free: *the unknown quantity $m \cdot s \bmod n$ currently lies inside $[2B, 3B)$* — a strong constraint on m , since s is chosen and known.

Putting the two together. Each oracle query is a free membership test — “does m times my chosen s land in this known narrow band?” — and each YES answer eliminates

most of the remaining candidates for m . Repeating this adaptively, choosing each new s based on everything learned so far, behaves like a binary search over the space of possible m : not quite halving the candidate set every query (the geometry is messier than a clean bisection), but shrinking it steadily and predictably until only one value of m survives. This is a classic **adaptive chosen-ciphertext attack**: it uses the decryption oracle itself, not any weakness in the RSA problem.

3.1.2 The Attack, at a High Level

Algorithm 7 Bleichenbacher’s Attack (structure, not full case analysis)

- 1: **Input:** target ciphertext $c = m^e \bmod n$; oracle $\text{Valid}(c') \in \{0, 1\}$ testing PKCS#1 conformance
 - 2: **Step 1 (blinding search):** find the smallest s_1 such that $c \cdot s_1^e \bmod n$ passes the oracle
 - 3: **Step 2 (interval narrowing):** maintain a shrinking set of intervals that must contain m ; each oracle query on a candidate s_i either confirms an interval or splits it further, using the known structure $[2B, 3B)$
 - 4: **Step 3 (interval refinement):** once a single interval remains, search increasingly precise values of s to shrink the interval toward a single point
 - 5: **Return** the unique m remaining once the interval has width 1
-

Reading the algorithm against the two ingredients above. At every stage, the attacker maintains a set of intervals guaranteed to contain the true m (initially, the whole range $\{0, \dots, n - 1\}$). Each new query picks a candidate s_i (Ingredient 1: scale by a chosen, known factor) and asks the oracle whether $ms_i \bmod n$ lands in $[2B, 3B)$ (Ingredient 2: the one-bit membership test). A YES answer means: for whichever interval m was in, only the sub-range consistent with $ms_i \in [2B, 3B)$ survives — everything else is provably ruled out and discarded. Repeating this steadily shrinks the total surviving range, exactly like narrowing down a hidden number through a sequence of “is it bigger than / smaller than” questions, until only a single value of m remains consistent with every answer received so far.

Remark 3.1.3 (Why “million-message attack”). *Bleichenbacher’s original paper estimated roughly 2^{20} (about a million) oracle queries to fully decrypt one ciphertext against 1024-bit RSA-PKCS#1v1.5 — expensive but entirely practical for an attacker with network access to a server that leaks the oracle bit, e.g. via a distinguishable TLS alert message (“bad padding” vs. other decryption failures). Later refinements (Bardou et al., 2012; ROBOT, 2017) reduced the required query count and revealed the attack was still exploitable in production TLS stacks nearly 20 years after publication — the flaw is in the error-handling contract of the protocol, which is easy to get subtly wrong even when the padding format itself is implemented correctly.*

Practical consequence. Never let decryption failure reasons be distinguishable to the attacker — not via error message content, not via response timing, not via connection behavior. The historically recommended mitigation for legacy PKCS#1 v1.5 is to proceed *as if* decryption succeeded on failure (substituting random plaintext) so that no observable signal exists, rather than trying to enumerate and suppress every possible error channel. The complete fix, however, is to stop using PKCS#1 v1.5 for new designs and use OAEP (Section 3.6), whose security proof directly rules out this class of attack.

3.2 Timing Attacks

3.2.1 The Leakage Source

Recall the square-and-multiply algorithm from Lesson 1: the `if` branch executes an extra multiplication exactly when the current secret exponent bit is 1. Even without a visible branch, the multiplication and squaring operations themselves can take data-dependent time (due to conditional reduction steps, cache access patterns, or hardware multiplier behavior on operands of different sizes/structure).

Kocher’s Timing Attack (1996), core idea. An attacker who can submit many ciphertexts $c_1, c_2, \dots$ to a server and measure the wall-clock time T_i each decryption/signature takes can recover the secret exponent d **one bit at a time**, from the most significant bit downward. Having correctly guessed the top j bits of d , the attacker can *simulate* the modular exponentiation up to that point for each c_i and correlate the simulated intermediate computation’s cost with the real measured T_i . A correct bit guess produces a statistically detectable correlation (the simulated step matches what the real hardware actually did); a wrong guess does not. Repeating bit by bit, with enough samples per bit to beat measurement noise, recovers all of d .

Why this works even though a single timing measurement is far too noisy to read a bit off directly. A single decryption’s total time T_i is the sum of contributions from *every* step of the exponentiation — thousands of squarings and conditional multiplications — plus random measurement noise (network jitter, OS scheduling, cache state left over from other processes). No single measurement isolates the cost of one specific bit’s decision. The trick is to use *many* ciphertexts and *group them by prediction*: for a guessed value of the next bit, simulate whether the extra multiplication should occur for each c_i , and split the sample into two groups — “simulation predicts extra multiply” versus “simulation predicts no extra multiply.” If the guessed bit is correct, the first group’s real measured times T_i will be, *on average*, very slightly higher than the second group’s — because they really did perform one more multiplication each — and this tiny average difference becomes statistically detectable once enough samples are averaged together, the same way a coin’s small bias becomes obvious after enough flips even though any single flip tells you nothing. If the guessed bit is wrong, the simulation’s grouping is uncorrelated with what the hardware actually did, so both groups’ average times come out statistically indistinguishable. This is precisely why the attack proceeds strictly one bit at a time, most-significant first: correctly recovering bit j requires the simulation of steps up to bit j to already be accurate, which needs every higher-order bit to already be known.

Remark 3.2.1 (Why this generalizes well beyond the toy square-and-multiply). *The same statistical idea applies to any implementation where some intermediate computational step’s cost depends on secret bits — including CRT-RSA’s d_p, d_q exponentiations (arguably a richer target, since each exponent is only half the length, so fewer bits need to be recovered per correlated measurement), and even to cache-timing variants that measure which memory lines were touched rather than wall-clock time directly (e.g., a colocated attacker on the same physical machine observing cache eviction patterns).*

Example 3.2.2 (Toy illustration of the bit-by-bit correlation). *Let the secret exponent be the 6-bit toy value $d = 101101_2 = 45$, used with right-to-left square-and-multiply (Lesson 1, Algorithm 2). Processing d ’s bits from the least-significant end, the algorithm performs a squaring every step, plus an extra multiplication exactly on the steps where the bit is 1: reading $d = 101101_2$ least-significant-bit first gives the bit sequence 1, 0, 1, 1, 0, 1, so extra multiplications occur on steps 1, 3, 4, 6 and are skipped on steps 2, 5. An attacker who has correctly guessed the first few (low-order) bits can simulate exactly which steps should include the extra multiplication; timing many real executions and checking whether the measured cost is high (extra multiply present) or low (skipped) on the predicted step is precisely the one-bit correlation test Kocher’s*

attack performs, repeated bit by bit until all 6 bits of d are recovered. At real key sizes (d having thousands of bits, not 6), each bit requires many timing samples to beat measurement noise, but the mechanism is identical to this toy trace.

3.2.2 Defense: Blinding

Definition 3.2.3 (RSA blinding). Before computing $m = c^d \bmod n$, the signer/decryptor picks a fresh random r (coprime to n) for every operation, computes

$$c' = c \cdot r^e \bmod n, \quad m' = (c')^d \bmod n, \quad m = m' \cdot r^{-1} \bmod n.$$

Proposition 3.2.4 (Correctness of blinding). $m' = (c \cdot r^e)^d = c^d \cdot r^{ed} \equiv c^d \cdot r \pmod{n}$ (using $r^{ed} \equiv r \pmod{n}$ by the same Euler's-theorem identity from Lesson 1 that makes RSA correct in the first place), so $m' \cdot r^{-1} \equiv c^d \equiv m \pmod{n}$.

Why blinding defeats the timing attack. The exponentiation the attacker times is now $(c')^d \bmod n$ for a value c' that the attacker cannot predict or control (it is masked by a fresh random r chosen internally, unknown to the attacker, on every single call). Since the attacker cannot correlate the timing of a specific known input against a specific bit-guess simulation — the actual base being exponentiated is different and unknown every time — the statistical correlation Kocher's attack relies on collapses to noise. **Blinding does not make the exponentiation itself constant-time**; it removes the attacker's ability to choose or know the input being timed, which is what the attack actually needs.

Example 3.2.5 (Blinding, fully worked). Reuse the familiar toy key $p = 61, q = 53, n = 3233, e = 17, d = 2753$. An attacker sends ciphertext $c = 65^{17} \bmod 3233 = 2790$ (encryption of $m = 65$) and wants to time the server's decryption of exactly this c .

The server instead picks a fresh random blinding factor $r = 41$ (coprime to n) and computes

$$c' = c \cdot r^{17} \bmod 3233 = 2130.$$

It is $c' = 2130$ — not the attacker's original $c = 2790$ — whose decryption the server actually times: $m' = (c')^d \bmod n = 2130^{2753} \bmod 3233 = 2665$. Unblinding recovers the true answer:

$$m = m' \cdot r^{-1} \bmod n = 2665 \cdot 41^{-1} \bmod 3233 = 65,$$

correctly matching the original message. **The attacker's timing measurement is on $c' = 2130$, a value they never chose and cannot predict** — a fresh random r is drawn on every call, so the same c is never timed twice under the same disguise, and no bit-guess simulation the attacker builds around their chosen c can be correlated against what the server actually computed.

Practical consequence. Production RSA implementations combine *two* defenses, not one: (1) blinding, to remove attacker control over the timed input, and (2) constant-time modular exponentiation (e.g., Montgomery ladder instead of naive square-and-multiply, avoiding secret-dependent branches or table lookups entirely) as defense in depth, since blinding alone does not stop attackers who can observe *which physical operations occurred* rather than just aggregate time (e.g., some cache/microarchitectural attacks). Real-world CVEs against RSA timing side channels (e.g., against OpenSSL's Montgomery multiplication in various years) have repeatedly targeted implementations that had blinding but incomplete constant-time arithmetic underneath it.

3.3 Fault Attacks: The Bellcore Attack on CRT-RSA

3.3.1 Setup

Recall CRT-RSA decryption/signing from Lesson 1 (Section 1.2.3): the signer computes

$$s_p = c^{d_p} \bmod p, \quad s_q = c^{d_q} \bmod q,$$

and recombines via CRT to get the full signature s . This is roughly $4\times$ faster than computing $c^d \bmod n$ directly — which is precisely why real implementations use it, and precisely what makes the following attack possible.

Why this should feel dangerous, before the proof. Recall from Lesson 1 exactly what CRT guarantees: the map $x \mapsto (x \bmod p, x \bmod q)$ is a perfect one-to-one correspondence — every pair of remainders corresponds to *exactly one* x modulo n . CRT-RSA signing builds a value in exactly the reverse direction: it computes two remainders (s_p, s_q) separately and *trusts* that recombining them gives the one true x that has those two remainders. That trust is precisely what a fault attack violates. If a hardware glitch corrupts only s_p , the recombination step goes on trusting it anyway and confidently reconstructs *a* number with the given pair of remainders — just not the number anyone intended. The faulty output is CRT working perfectly on a garbage input, and the very same reconstruction formula that recombines correct partial results also, unknowingly, discloses exactly where the two halves disagree.

Theorem 3.3.1 (Bellcore attack, Boneh–DeMillo–Lipton, 1997). *Suppose an attacker can induce a single transient computational fault (via voltage glitching, clock glitching, laser fault injection, or simply naturally-occurring hardware error) during exactly one of the two CRT branches — say the s_p computation — while the s_q branch computes correctly, and can obtain the resulting faulty signature s' on a message m whose value the attacker knows. Then the attacker recovers the prime factor q (and hence the full private key) in polynomial time, using only s' , m , and the public (n, e) — **no correct signature is needed at all.***

Proof, step by step. Step 1: pin down what the fault does and does not change. The q -branch computation was untouched by the fault, so the faulty output s' still agrees with the correct signature modulo q : $s' \equiv m^d \pmod{q}$. The p -branch was corrupted, so s' disagrees with the correct signature modulo p : $s' \not\equiv m^d \pmod{p}$ (a random hardware fault essentially never happens to land on a value that accidentally still satisfies the same congruence).

Step 2: translate “agrees mod q , disagrees mod p ” into a divisibility statement about $s'^e - m$. Correctness of RSA (Lesson 1, Section 1.3) says that for the *true* signature $s = m^d$, we’d have $s^e \equiv m \pmod{n}$, and in particular $s^e \equiv m \pmod{q}$ and $s^e \equiv m \pmod{p}$ individually. Since $s' \equiv m^d \pmod{q}$ still holds (Step 1), the same correctness argument restricted to mod q gives $s'^e \equiv m \pmod{q}$, i.e.

$$q \mid (s'^e - m).$$

But $s' \not\equiv m^d \pmod{p}$, so the parallel argument mod p *fails*: $s'^e \not\equiv m \pmod{p}$, i.e.

$$p \nmid (s'^e - m).$$

Step 3: extract q with a single gcd. Step 2 says q divides $s'^e - m$ while p does not — exactly the situation where computing $\gcd(s'^e - m \bmod n, n)$ isolates q cleanly: any common factor of $s'^e - m$ and $n = pq$ can only involve q (since p does not divide $s'^e - m$ at all), and q itself certainly does divide both. So

$$\gcd(s'^e - m \bmod n, n) = q.$$

Step 4: finish factoring and rebuild the private key. With q in hand, compute $p = n/q$ by ordinary integer division, then $\varphi(n) = (p-1)(q-1)$, then $d = e^{-1} \bmod \varphi(n)$ exactly as in ordinary key generation (Lesson 1) — the entire private key falls out from one faulty signature and one gcd computation. $\square$

The one-sentence summary. A fault attack on CRT-RSA is CRT’s own uniqueness guarantee turned into a weapon: corrupting exactly one branch produces a number that is quietly “correct modulo q , wrong modulo p ” rather than uniformly wrong, and that asymmetry is precisely what a single gcd computation detects and exploits.

Example 3.3.2 (Bellcore attack, fully worked). *Using the familiar toy key $p = 61, q = 53, n = 3233, e = 17, d = 2753$, with $d_p = 53, d_q = 49, q^{-1} \bmod p = 38$. The signer correctly computes $s_p = 65^{53} \bmod 61 = 4$ and $s_q = 65^{49} \bmod 53 = 12$, recombining to the correct signature $s = 588$ (indeed $588^{17} \bmod 3233 = 65 = m$, as required).*

Now suppose a fault corrupts the p -branch only, flipping s_p to $s'_p = 5$ (one off from the correct value) while $s_q = 12$ is untouched. Recombining via the same CRT formula gives a faulty signature $s' = 2602$. Checking: $(s')^{17} \bmod 3233 = 1496$, and indeed $1496 \equiv 12 \equiv m \pmod{53}$ (the q -branch congruence survives) while $1496 \equiv 32 \not\equiv 4 \equiv m \pmod{61}$ (the p -branch congruence is broken).

The attacker, knowing only $n = 3233, e = 17$, the message $m = 65$, and the single faulty signature $s' = 2602$, computes

$$\gcd((s')^e \bmod n - m, n) = \gcd(1496 - 65, 3233) = \gcd(1431, 3233) = \mathbf{53} = q.$$

A single faulty signature factored the modulus completely — recovering q , then $p = 3233/53 = 61$, then $\varphi(n)$ and d itself. No correct signature was ever needed.

Practical consequence: signature verify-before-release. The standard defense is procedural, not algebraic: **always verify a freshly computed CRT-RSA signature against the public key before releasing it** — i.e. check $s^e \bmod n \stackrel{?}{=} m$ internally, and discard/retry silently if the check fails, releasing nothing. A genuine fault will (overwhelmingly likely) fail this check, so the faulty signature the attacker needs is simply never emitted. This single check closes the entire attack class at negligible extra computational cost (one public-exponent exponentiation, which is cheap since e is small, e.g. 65537), and is mandated by essentially every modern smart-card and HSM signing standard.

Remark 3.3.3 (Why this specifically targets CRT-RSA, and why that matters). *Non-CRT RSA (computing $c^d \bmod n$ directly) is also vulnerable to fault attacks in principle, but recovering the key from a single faulty full-size exponentiation is far harder — there is no clean two-branch structure to exploit algebraically. This is precisely the trade-off flagged in Lesson 1: the $4\times$ CRT speedup is bought at the cost of a much sharper fault-attack surface, which is why smart cards and HSMs that use CRT-RSA must implement the verify-before-release countermeasure (or an equivalent redundant computation), while implementations that skip CRT entirely face a smaller (but nonzero) fault-attack risk.*

3.4 ROCA: When Key Generation Itself Is the Vulnerability

3.4.1 Background

ROCA (“Return of Coppersmith’s Attack,” Nemec et al., 2017) is not a flaw in RSA’s mathematics or in a protocol’s error handling — it is a flaw in a specific *key-generation implementation* (the Infineon RSALib used in many smart cards, TPMs, and government-issued ID cards) that made factoring *practically feasible* for affected keys, at sizes (512–2048 bits) that are otherwise completely safe against GNFS.

The vulnerable construction. The affected library generated primes of the special form

$$p = k \cdot M + (65537^a \bmod M)$$

for a secret integer k , a secret exponent a , and a fixed public constant M (a product of the first several hundred small primes — a “primorial”), apparently as a performance optimization for the prime-generation process on constrained smart-card hardware. This construction was undisclosed until reverse-engineered by researchers.

Why this is a coset, and why a coset is much smaller than it looks. Fix M and think of all integers as sorted into M buckets by their remainder mod M — bucket 0, bucket 1, ..., bucket $M - 1$. An ordinary randomly-generated prime p could fall in essentially any bucket (any residue coprime to M). The vulnerable construction instead *forces* p ’s bucket to be one of the specific values $65537^a \bmod M$ for some integer a — and because 65537 has some finite multiplicative order modulo M (Lesson 1’s group-order idea, applied to the group $(\mathbb{Z}/M\mathbb{Z})^*$), the set of values $65537^a \bmod M$ reachable as a ranges over all integers is not all of $\{0, \dots, M - 1\}$ — it cycles through only $\text{ord}_M(65537)$ distinct residues before repeating. So instead of p ’s bucket being one of M possibilities, it is secretly one of only $\text{ord}_M(65537)$ possibilities — a tiny, explicitly listable subset, even though M itself is astronomically large.

Proposition 3.4.1 (Why this collapses the search space). *An attacker who knows the public modulus $n = pq$ and suspects it was generated this way does not need to search all $\approx \sqrt{n}$ possible values of p . Instead, since $p \bmod M$ is one of only $\text{ord}_M(65537)$ possible residues (the multiplicative order of 65537 modulo M — typically a number in the thousands to low millions for the real ROCA parameters, versus M itself and $\sqrt{n}$ being astronomically larger), the attacker can:*

1. Enumerate the (comparatively tiny) set of candidate residues $r_a = 65537^a \bmod M$.
2. For each candidate residue, use **Coppersmith’s method** (Lesson 2, Section 2.6) to search for a small k completing $p = kM + r_a$ into an actual factor of n — Coppersmith’s small-root machinery is exactly suited to this “most of p is unknown but constrained to a coset” structure.

This turns a brute-force search over all of $\mathbb{Z}$ into a brute-force search over a small candidate set, composed with a polynomial-time root-finding step per candidate — dropping the effective attack cost for affected 1024-bit keys from “infeasible” to a few CPU-hours, and for affected 2048-bit keys to a few CPU-months on modest hardware (the original paper’s reported figures), rather than the millennia GNFS would require.

Why Coppersmith’s method specifically is the right tool for Step 2. Fixing a candidate residue r_a turns “find p ” into “find the unknown k in $p = kM + r_a$ ” — and if p has, say, 1024 bits while M has around 200 bits, then k has only around $1024 - 200 = 824$ bits: a large fraction of p (the residue $r_a \bmod M$) is already pinned down, leaving only a comparatively *small* unknown k to find. This is exactly the “small root of a polynomial mod n ” shape Section 2.6 in Lesson 2

was built for: the polynomial $f(x) = kM + r_a - x$ (thinking of x as standing in for the unknown residual bits) has a small root precisely when the coset construction has pinned down enough of p 's bits, which ROCA's fingerprint structure guarantees.

Example 3.4.2 (Toy illustration of the fingerprint structure — not real ROCA scale). *Let the (deliberately tiny, illustrative-only) constant be $M = 3 \cdot 5 \cdot 7 \cdot 11 = 1155$. Compute $65537 \bmod M = 857$; the multiplicative order of 857 modulo 1155 turns out to be 12 — meaning there are only **12** distinct residues $65537^a \bmod M$ ever reachable, out of the 1155 residues mod M in general (and utterly negligible next to $\sqrt{n}$ for a cryptographic-size n). Taking $a = 1$ (so the residue is $65537 \bmod 1155 = 857$) and $k = 2$ gives the candidate*

$$p = 2 \cdot 1155 + 857 = 3167,$$

which is in fact prime. An attacker facing a modulus $n = p \cdot q$ built this way would not need to guess p among all integers near $\sqrt{n}$ — only among the 12 residues mod 1155 (here, $\{1, 331, 362, 529, 593, 692, 694, 857, 923, 991, 1024, 1088\}$), each followed by a Coppersmith-style search for k . At real ROCA parameters (M a $\sim 2^{200}$ -ish primorial, order in the low millions), the identical structural idea — not this exact toy arithmetic — is what makes the real attack practical.

Practical consequence. This vulnerability class does not indicate any weakness in RSA itself, in Coppersmith's method, or in any of the earlier lectures' key-generation advice (large random independent primes, avoiding smooth $p - 1$, avoiding shared/related moduli) — it is precisely what happens when an implementation *silently deviates* from that advice for an unstated performance reason. The lesson for practitioners: **never trust that “a widely deployed, certified library” generates primes uniformly at random** without independent verification; ROCA affected certified smart cards and government ID infrastructure for years before detection, precisely because the deviation was invisible from the public key alone unless someone specifically tested for the fingerprint.

3.5 Side-Channel Countermeasures: A Consolidated Checklist

The individual defenses mentioned throughout this lecture form a coherent set of standard practices. Collecting them here as a reference:

Threat	Countermeasure	Covered in
Padding-oracle CCA	OAEP instead of PKCS#1 v1.5; uniform error handling	Sec. 1, Sec. 6
Timing leakage	RSA blinding (randomize the input)	Sec. 2
Timing/cache leakage	Constant-time modular exponentiation (Montgomery ladder)	Sec. 2
CRT fault injection	Verify signature before release; redundant computation	Sec. 3
Weak key generation	Independent verification of prime-generation randomness; avoid undisclosed proprietary shortcuts	Sec. 4
General implementation drift	Use vetted, widely-audited cryptographic libraries; do not hand-roll modular exponentiation or padding logic	All sections

Remark 3.5.1 (The unifying theme, one more time). *Every defense above targets the gap between the mathematical specification of RSA and its physical realization. Lessons 1–2 taught you when RSA is mathematically sound; this lecture teaches you that mathematical soundness is necessary but nowhere near sufficient — an implementation can be simultaneously “textbook-perfect” in its algebra and completely broken in its execution.*

3.6 OAEP: A Security Sketch

3.6.1 Construction

Optimal Asymmetric Encryption Padding (Bellare–Rogaway, 1994) replaces the fixed, structured PKCS#1 v1.5 padding with a randomized, “all-or-nothing”-looking transform, before the result is encrypted with textbook RSA.

The design goal, before the formulas. Every attack in Lessons 2–3 up to this point succeeded because *some* structure survived from the plaintext into what actually got RSA-encrypted: a small message, a predictable format, a known relationship between two messages, a cheaply-checkable validity condition. OAEP’s goal is to erase all of that: take the message and a fresh random seed, and mix them together so thoroughly that the result is indistinguishable from a uniformly random string to anyone who does not know the seed — while still being perfectly and uniquely invertible by someone who does.

OAEP encoding (simplified). To encode a message m (padded with a fixed-length string of zero bits PS for domain separation) using two hash functions G (a mask-generating function) and H , and a fresh random seed r :

$$X = (m \parallel \text{PS}) \oplus G(r), \quad Y = r \oplus H(X), \quad \text{EM} = X \parallel Y,$$

then EM (interpreted as an integer $< n$) is encrypted as ordinary textbook RSA: $c = \text{EM}^e \bmod n$. Decryption inverts each XOR step in reverse (recovering r from Y and $H(X)$, then $m \parallel \text{PS}$ from X and $G(r)$) and **rejects** if the recovered padding string does not exactly match the expected PS.

Why two rounds of masking, and why in this order. A single XOR mask ($X = m \oplus G(r)$ alone) would already hide m from anyone who does not know r — but the seed r itself would then have to travel alongside X in the clear for the recipient to reconstruct anything, defeating the purpose entirely. OAEP’s second step, $Y = r \oplus H(X)$, hides r *using X itself as the mask key* — so the seed is never exposed either. This creates a tight two-way dependency: Y cannot be computed without already knowing X , and decoding X needs r , which needs Y and X together. That interlocking is exactly what makes a single flipped bit anywhere in EM cascade into an unpredictable, unrelated-looking recovered message on decryption (rather than a small, predictable change) — precisely the property that shuts down chosen-ciphertext manipulation.

Example 3.6.1 (Toy illustration of the OAEP masking mechanism — not a real hash). *The real G, H are cryptographic hash functions; to see the seed/mask mechanics with actual numbers, use toy 4-bit stand-ins $G(r) = (7r + 3) \bmod 16$ and $H(X) = (5X + 1) \bmod 16$ (fixed, public functions — not secret), omit PS for brevity, and encode the toy message $m = 5$ with fresh random seed $r = 9$:*

$$G(9) = 2, \quad X = m \oplus G(r) = 5 \oplus 2 = 7, \quad H(7) = 4, \quad Y = r \oplus H(X) = 9 \oplus 4 = 13.$$

So $\text{EM} = (X, Y) = (7, 13)$. Decoding reverses the steps in the opposite order: $r' = Y \oplus H(X) = 13 \oplus 4 = 9$, then $m' = X \oplus G(r') = 7 \oplus 2 = 5$ — correctly recovering m .

*Now encrypt the same message $m = 5$ again with a fresh seed $r = 2$: $G(2) = 1$, so $X = 5 \oplus 1 = 4$; $H(4) = 5$, so $Y = 2 \oplus 5 = 7$; giving $\text{EM} = (4, 7)$ — completely different from $(7, 13)$ despite encoding the identical message. **This is exactly the property that defeats Håstad and Franklin–Reiter:** two encodings of the same (or related) plaintext no longer share any visible algebraic structure once OAEP has randomized them, before RSA is even applied.*

Security sketch, in the random oracle model. Modeling G and H as independent random oracles, Bellare and Rogaway showed OAEP-RSA achieves **IND-CCA2 security** (indistinguishability under adaptive chosen-ciphertext attack) whenever the underlying trapdoor permutation (here, RSA) is one-way.

The proof idea, unpacked. “ G, H are random oracles” means: the only way to learn anything about the output of G or H on a given input is to explicitly query them on that exact input — there is no shortcut, no formula to exploit, only lookups. Suppose an adversary could distinguish OAEP encryptions of two messages, or extract useful information from a decryption-oracle query on a chosen ciphertext. Either capability requires the adversary to have gotten *some* information correlated with the true seed r or the true masked value X underlying a target ciphertext $c = \text{EM}^e \bmod n$ — and because G, H are random oracles, the *only* way to obtain such correlated information is to have already queried G or H on that specific r or X . But querying G or H on the value tied to c ’s RSA pre-image is, itself, morally equivalent to already knowing $\text{EM} = c^d \bmod n$ — i.e., to already having inverted RSA on c . So a simulator who watches the adversary’s oracle queries can extract an RSA inverse from whichever query turns out to be the “real” one. Informally: **any successful attack on OAEP-RSA can be converted into an algorithm that inverts plain RSA**, which we assume is hard.

Remark 3.6.2 (Why OAEP specifically defeats the earlier attacks). *Each attack in Lessons 2–3 relied on some algebraic or structural regularity surviving into the encrypted value:*

- **Håstad / Franklin–Reiter** (Lesson 2) needed related plaintexts to remain related after encoding. OAEP’s random seed r makes two encryptions of related (or even identical) messages look like uniformly random, unrelated strings before RSA is even applied — there is no longer any algebraic relationship for CRT or polynomial-GCD tricks to exploit.
- **Bleichenbacher’s padding oracle** (Section 1) exploited the fact that a large fraction of PKCS#1v1.5-decrypted garbage still “looks like” a differently-shaped valid message with nonzero probability, and that validity itself was cheaply, deterministically checkable and distinguishable. OAEP’s redundancy check (the PS match) has negligible probability of accidentally validating on a random/incorrect decryption, and — critically — OAEP’s security proof accounts for a decryption oracle directly (IND-CCA2 is defined with exactly such an oracle available to the adversary), which PKCS#1v1.5’s original design did not.
- **Malleability and determinism** (flagged already in Lesson 1) are both eliminated: the random seed r makes encryption probabilistic, and the tightly-coupled XOR/hash structure means a modified ciphertext decrypts (if it decrypts at all) to something the redundancy check almost always rejects, rather than to a predictably related plaintext.

What OAEP does *not* fix. OAEP is a defense at the *mathematical/protocol* layer. It does nothing by itself against Section 2’s timing attacks or Section 3’s fault attacks against the underlying modular exponentiation — those require blinding, constant-time arithmetic, and verify-before-release, independently of which padding scheme sits on top. Real-world security requires *all* of these layered together; OAEP alone is necessary but not sufficient.

3.7 Guillou–Quisquater: An RSA-Based Zero-Knowledge Proof

3.7.1 Motivation: Proving Identity Without Revealing the Secret

Everything so far in this RSA module has been about encryption or signing. Guillou–Quisquater (1988) uses the same algebraic structure — an RSA-type modulus and modular exponentiation — for a different goal: letting a prover convince a verifier “*I know a secret s tied to my claimed identity*” without revealing s , and without letting the verifier (or an eavesdropper) later impersonate the prover.

What a protocol like this has to achieve, in plain terms, before any algebra. Think of three separate promises a scheme like this must keep simultaneously, each of which fails for a naive approach:

- **Completeness:** someone who genuinely knows the secret should always be able to convince the verifier. (Trivial to achieve alone — just reveal the secret directly.)
- **Soundness:** someone who does *not* know the secret should have essentially no chance of convincing the verifier. (Also easy alone — just never let anyone in without proof.)
- **Zero-knowledge:** the verifier should learn *nothing* about the secret itself from watching the proof, other than the one bit “this person knows it.” (This is the hard one — the whole reason a protocol is needed rather than simply typing in a password, which *does* reveal the secret to whoever is listening.)

The tension is between soundness and zero-knowledge: soundness wants the prover to demonstrate real, specific knowledge of s , while zero-knowledge wants that demonstration to leak nothing reusable. Guillou–Quisquater resolves this tension with a three-message **commit/challenge/response** structure: the prover commits to something *before* seeing what will be asked, is then asked an unpredictable question, and answers in a way that only someone who really knows s could answer correctly for an unpredictable question — without the answer itself ever containing s .

Setup. A trusted authority publishes an RSA-like modulus $n = pq$ and a public verification exponent v (coprime to $\varphi(n)$, exactly as with an RSA public exponent). Each prover’s identity is mapped, via a public redundancy/hash function, to a value J ; the authority (who alone knows $\varphi(n)$, i.e. p, q) issues the prover a secret credential s satisfying

$$s^v \cdot J \equiv 1 \pmod{n}, \quad \text{i.e.} \quad s \equiv J^{-1/v} \pmod{n},$$

computable by the authority (who can take v -th roots mod n using $\varphi(n)$) but not by anyone else, exactly as RSA decryption requires the private exponent.

Algorithm 8 Guillou–Quisquater Identification Protocol (one round)

- 1: **Prover** picks random r coprime to n , sends commitment $x = r^v \bmod n$
 - 2: **Verifier** picks random challenge $c \in \{0, 1, \dots, v-1\}$, sends c
 - 3: **Prover** computes and sends $y = r \cdot s^c \bmod n$
 - 4: **Verifier** accepts iff $x \equiv y^v \cdot J^c \pmod{n}$
-

Reading the protocol against the three goals above. The commitment $x = r^v \bmod n$ is sent *before* the challenge c is even chosen — the prover cannot tailor x to a challenge they have not seen yet. The response $y = r \cdot s^c \bmod n$ blends the fresh randomness r with the secret s raised to the (unpredictable, just-revealed) challenge power — a genuine one-time mixture, not a static reveal of s . The verification equation checks that the blend is consistent with *some* valid s , without ever recomputing or isolating s itself.

Proposition 3.7.1 (Completeness). *If the prover genuinely knows s and follows the protocol, $y^v \cdot J^c = (rs^c)^v \cdot J^c = r^v \cdot s^{vc} \cdot J^c = x \cdot (s^v J)^c \equiv x \cdot 1^c \equiv x \pmod{n}$, so an honest verifier always accepts.*

Proposition 3.7.2 (Soundness, sketch). *A cheating prover who does not know s but wants to pass for a specific challenge value c can indeed fake a valid-looking transcript for that one value of c (by choosing y first, then setting $x = y^v J^c \pmod{n}$ to match) — but must commit to x before learning the verifier’s actual random challenge. A cheating prover can therefore satisfy at most one of the v possible challenge values with a given commitment x , so guessing the verifier’s challenge correctly happens with probability exactly $1/v$. Choosing v large (e.g. a value comparable in size to a standard RSA public exponent) drives the cheating probability negligibly low in a single round, unlike Fiat–Shamir-style protocols built on a $v = 2$ structure, which need many sequential rounds to reach the same soundness level.*

Why soundness needs the commitment to come first — the intuition behind Proposition 3.7.2. If the prover could see the challenge c before committing to x , cheating would be trivial: fix y arbitrarily, compute $x = y^v J^c \pmod{n}$ working backwards, and the check passes for that known c every time, with no knowledge of s required at all. The entire soundness guarantee rests on *ordering*: x is locked in first, c is only then revealed at random, and a fixed x algebraically corresponds to a valid response y for only *one* value of c (since y^v is a fixed function of x and c together, and v -th roots mod n are unique once they exist) — so a prover who guessed wrong before committing is simply stuck.

Remark 3.7.3 (Zero-knowledge, sketch). *A simulator that does not know s can still produce transcripts (x, c, y) indistinguishable from real ones without interacting with a real prover: pick c and y uniformly at random first, then set $x = y^v J^c \pmod{n}$ (exactly the “rewind and fake the commitment” trick used in Proposition 3.7.2, but now run by the simulator rather than an attacker). Since x is computed from uniformly random y, c , the resulting triple has exactly the same distribution as a genuine protocol run, which is the essence of the honest-verifier zero-knowledge argument: nothing in the transcript could not have been produced without knowledge of s , so the transcript itself leaks no information about s .*

Why the very same trick proves both soundness *and* zero-knowledge, and why that is not a contradiction. It looks paradoxical that “work backwards from (y, c) to fake x ” is simultaneously the cheating prover’s only strategy (Proposition 3.7.2) and the simulator’s entire proof technique (the remark above) — but the two use it in opposite roles. The cheating prover must guess c *before* the real verifier reveals it, and fails whenever the guess is wrong: this is what makes the protocol sound against a real, interactive verifier. The simulator, by contrast, is not trying to fool a live verifier in real time at all — it is only trying to produce a transcript that *looks* like a valid run when inspected afterward, and it is allowed to choose c itself as part of that simulated transcript. The same algebraic backwards-construction is a dead end for real-time cheating but a complete proof technique for showing that transcripts alone contain no extractable information about s .

Example 3.7.4 (Guillou–Quisquater, fully worked). *Reuse the familiar toy modulus $n = 3233$ ($p = 61, q = 53, \varphi(n) = 3120$), and take the public exponent $v = 7$ (check: $\gcd(7, 3120) = 1$, exactly the RSA-key-generation condition from Lesson 1). Suppose the authority has issued the prover secret $s = 100$ (with $\gcd(100, 3233) = 1$), corresponding to public identity value*

$$J \equiv s^{-7} \pmod{n} = 1818, \quad \text{check: } s^7 \cdot J \pmod{n} = 1.$$

Round: *the prover picks $r = 37$ and sends the commitment $x = 37^7 \pmod{3233} = 1700$. The verifier picks challenge $c = 3$. The prover, knowing $s = 100$, replies with $y = 37 \cdot 100^3 \pmod{3233} = 1548$.*

The verifier checks: $y^7 \cdot J^3 \bmod 3233 = 1548^7 \cdot 1818^3 \bmod 3233 = 1700 = x$. **Match** — the verifier accepts, and at no point did $s = 100$ itself appear in any message exchanged.

Example 3.7.5 (Why a cheating prover cannot do better than guessing). Suppose an impostor does not know s , but tries to cheat by anticipating the verifier’s challenge. Working backwards, they pick an arbitrary response $y = 222$ and a guessed challenge $c = 3$, then compute the commitment that would make the check pass for that guess:

$$x = y^7 \cdot J^3 \bmod 3233 = 937.$$

If the verifier happens to challenge with $c = 3$ (matching the guess), the check passes: $y^7 J^3 \bmod 3233 = 937 = x$. But if the verifier instead challenges with, say, $c = 5$, the same fixed $(x, y) = (937, 222)$ fails: $y^7 J^5 \bmod 3233 = 789 \neq 937$. Since x was committed before the challenge was revealed, the impostor had to guess c in advance — correctly with probability exactly $1/v = 1/7$, matching Proposition 3.7.2’s soundness claim precisely.

Remark 3.7.6 (The bridge back to earlier material). Guillou–Quisquater is structurally the RSA-flavored cousin of the Fiat–Shamir identification scheme (which uses $v = 2$ and a modulus whose factorization is likewise the trapdoor) — the underlying commit/challenge/response skeleton (Algorithm 8 above) is the same three-move structure used by essentially every modern zero-knowledge identification and signature scheme, whatever the underlying hard problem (factoring/RSA here; discrete log for Schnorr/Fiat–Shamir; lattice problems for schemes built the same way from LWE/SIS). Recognizing this shared skeleton — commit, challenge, response, verify — is the single most transferable idea to carry out of this lecture.

3.8 Summary

Summary

- **Padding oracles** (Bleichenbacher): a single bit of decryption-validity feedback, queried adaptively, fully decrypts PKCS#1v1.5 ciphertexts — the flaw is in error-handling, not in RSA’s algebra.
- **Timing attacks** (Kocher): secret exponent bits leak through execution time of modular exponentiation; blinding (randomizing the input) plus constant-time arithmetic are both required as defense.
- **Fault attacks** (Bellcore): a single corrupted CRT branch, combined with one known message/signature pair, factors n outright via a single gcd computation; verify-before-release closes this completely.
- **ROCA**: a real-world case where non-uniform, undisclosed key generation collapsed the effective search space enough for Coppersmith’s method (Lesson 2) to factor otherwise-safe key sizes.
- **OAEP**: randomized, redundancy-checked padding with an IND-CCA2 security proof (in the random oracle model) that directly rules out the algebraic and oracle-based attacks from this and the previous lecture.
- **Guillou–Quisquater**: the same RSA algebra, repurposed as a zero-knowledge identification protocol, with soundness error $1/v$ per round instead of $1/2$ — and structurally the same three-move skeleton as the lattice-based signatures earlier in the course.

The course-level takeaway. Across all three RSA lectures: correctness is a CRT argument (Lesson 1); security against *misuse* requires avoiding a specific list of structural traps (Lesson 2); and security in *deployment* requires an entirely separate discipline — constant-time arithmetic, blinding, fault detection, verified randomness in key generation, and provably secure padding (Lesson 3). None of these three layers substitutes for the others: a mathematically flawless, correctly-used RSA implementation can still be broken in milliseconds by a fault injector, and a side-channel-hardened implementation is still broken instantly if d is small or n is shared. Real-world RSA security is the conjunction of all three lectures, not any one of them alone.

Tutorial Exercises

1. In Bleichenbacher’s attack, explain concretely why the oracle needs only to distinguish “starts with 0x00 0x02” rather than fully validating the entire padding string, and why this makes the attack robust to minor implementation variations across different TLS stacks.
2. Suppose a timing attack has correctly recovered the top 6 bits of a 12-bit toy private exponent d used with square-and-multiply. Describe precisely what additional measurements the attacker needs to recover bit 7, and why the attack proceeds strictly most-significant-bit first.
3. Redo the Bellcore worked example with the fault instead corrupting the q -branch (s_q) rather than the p -branch, and show that the same gcd technique now recovers p instead of q .

4. Using $M = 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 = 15015$, compute the multiplicative order of $65537 \bmod M$, and compare the resulting candidate-residue count against M itself and against $\sqrt{n}$ for a realistic 2048-bit RSA modulus. What does this comparison tell you about why ROCA is practical while generic factoring is not?
5. Explain why OAEP's random oracle security proof does not, by itself, guarantee security against the timing and fault attacks earlier in this lecture. What does this imply about the completeness of a security argument that only addresses the mathematical padding layer?
6. Compare Guillou–Quisquater's soundness argument to Fiat–Shamir's. If a Fiat–Shamir-style scheme uses $v = 2$ per round and needs t rounds to reach soundness error 2^{-t} , how many G-Q rounds are needed to reach the same soundness error using $v = 2^{16} + 1 = 65537$? What communication-cost trade-off does this reveal?